

Lecture 21

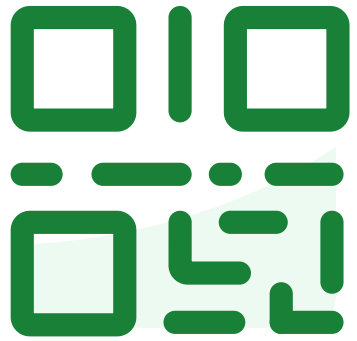
Transformers & LLMs

Transformers and Causal Language Modeling

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#9688650**

① The Slido app must be installed on every computer you're presenting from

slido

Recap – Attention

Inner-Product Search for Tokens

The **inner-product** between $\mathbf{Q}_n$ and $\mathbf{K}_i$ encodes the **relevance** of token i to computing the **representation** of token n .

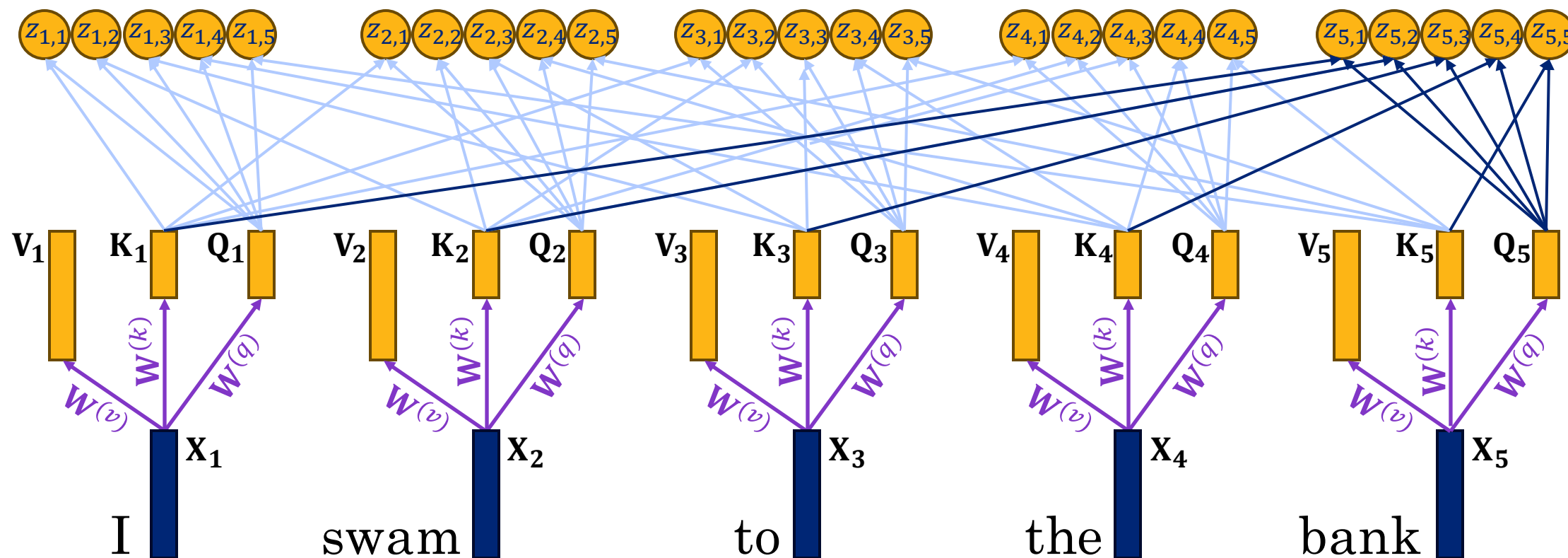
Symbols:

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$



Softmax Inner-Product Self-Attention

By computing the **softmax** along the rows of **Z** obtain the **attention matrix** $\alpha \in \mathbb{R}^{N \times N}$

Symbols:

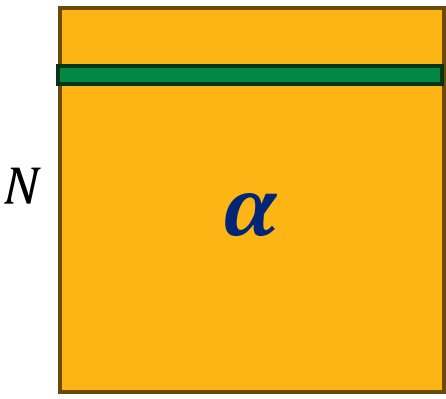
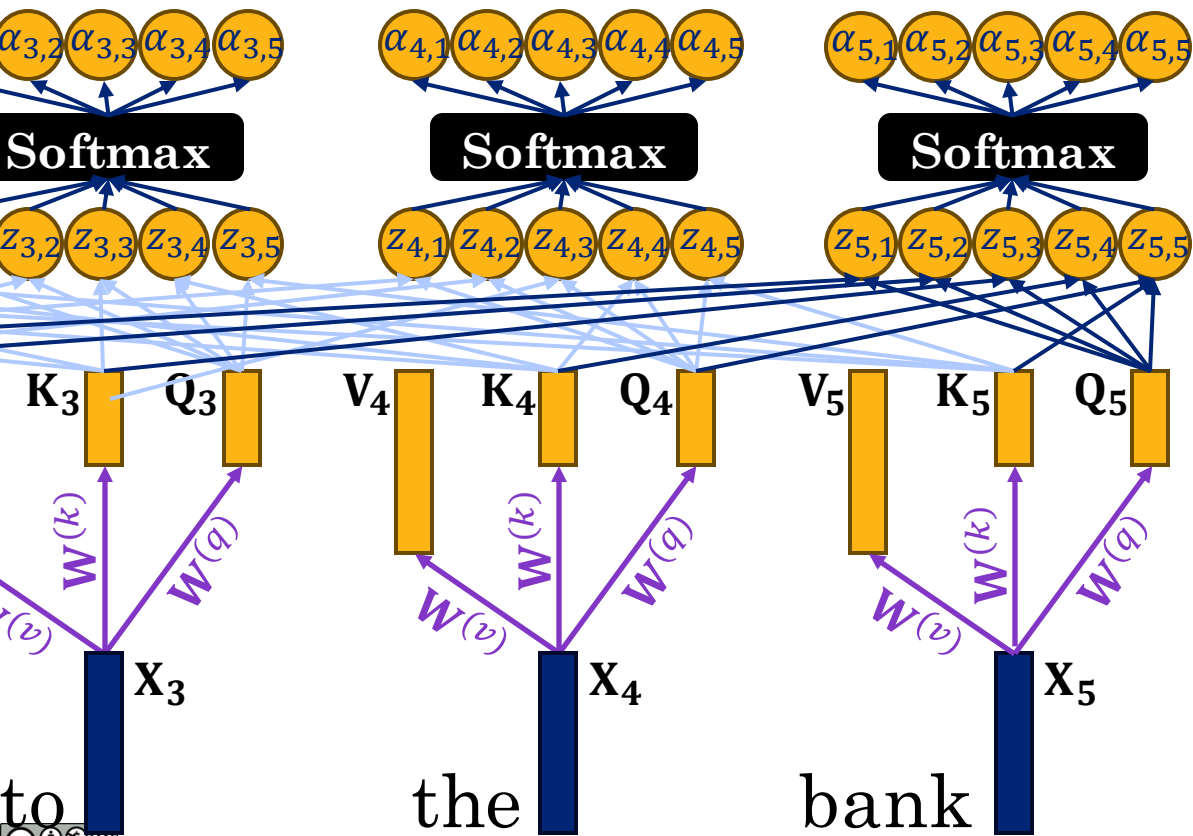
$$\begin{aligned} \mathbf{X} &\in \mathbb{R}^{N \times D} \\ \mathbf{V} &= \mathbf{X} \mathbf{W}^{(v)} \\ \mathbf{K} &= \mathbf{X} \mathbf{W}^{(k)} \\ \mathbf{Q} &= \mathbf{X} \mathbf{W}^{(q)} \end{aligned}$$



$$\mathbf{Z} = \mathbf{Q} \mathbf{K}^\top$$

$$\alpha = \text{SoftMaxRow}[\mathbf{Z}]$$

$$\alpha_{ni} = \frac{\exp(\mathbf{Z}_{ni})}{\sum_{j=1}^N \exp(\mathbf{Z}_{nj})}$$



Each row has
no negative entries
and sums to 1

The Attention Layer

$$\text{Attn}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = (\boldsymbol{\alpha} \mathbf{V})_n = \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

Symbols:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

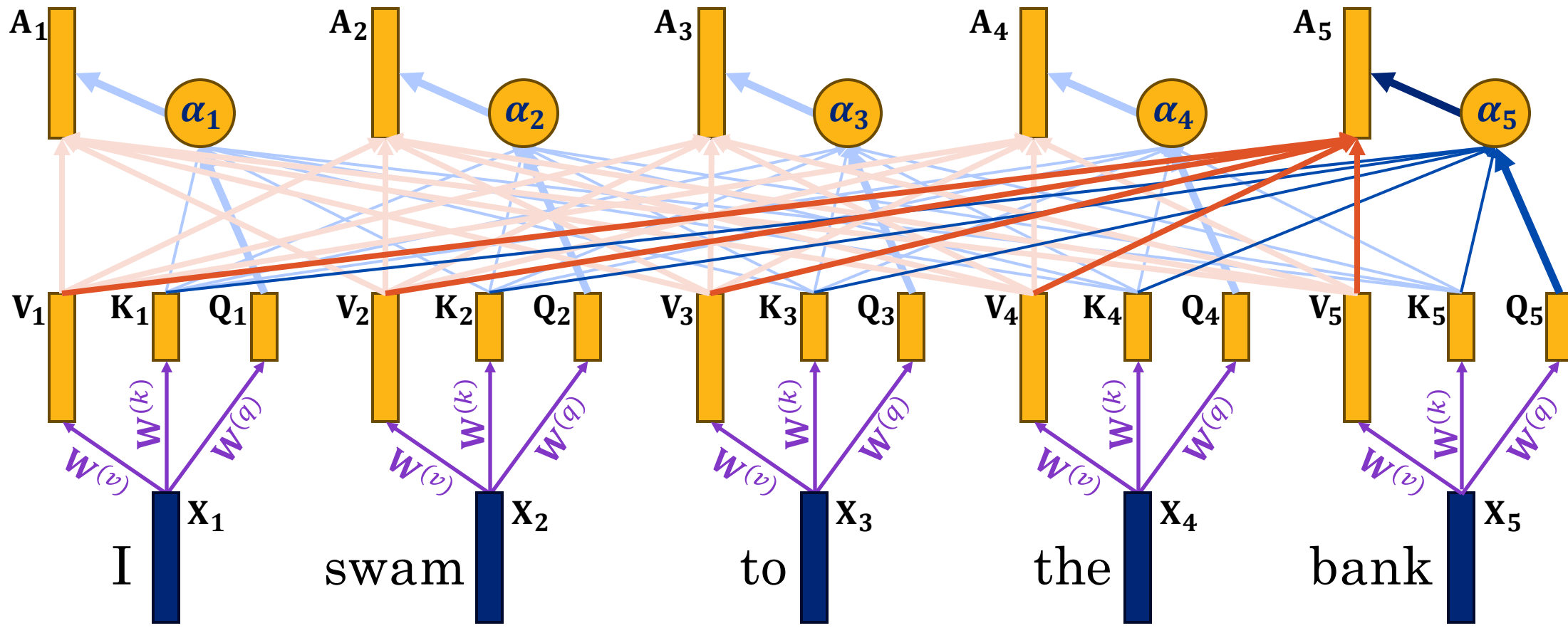
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\boldsymbol{\alpha} \in \mathbb{R}^{N \times N}$$



9688650





9688650

Demo

Self Attention in PyTorch

```
def scaled_dot_product_attention(Q: torch.Tensor,
                                K: torch.Tensor,
                                V: torch.Tensor):
    """
    Q: matrix of shape (B, N, d_k)
    K: matrix of shape (B, N, d_k)
    V: matrix of shape (B, N, d_v)
    """
    (B, N, d_k) = Q.shape
    (_, _, d_v) = V.shape
    K_T = K.transpose(-2, -1) # (B, d_k, N)
    dot_product = (Q @ K_T) / (d_k ** 0.5) # (B, N, N)

    attention = F.softmax(dot_product, dim=-1) # (B, N, N)
    return attention @ V # (B, N, N) * (B, N, d_v) = (B, N, d_v)
```

✓ 0.0s

[Pytorch](#)
[Implementation](#)

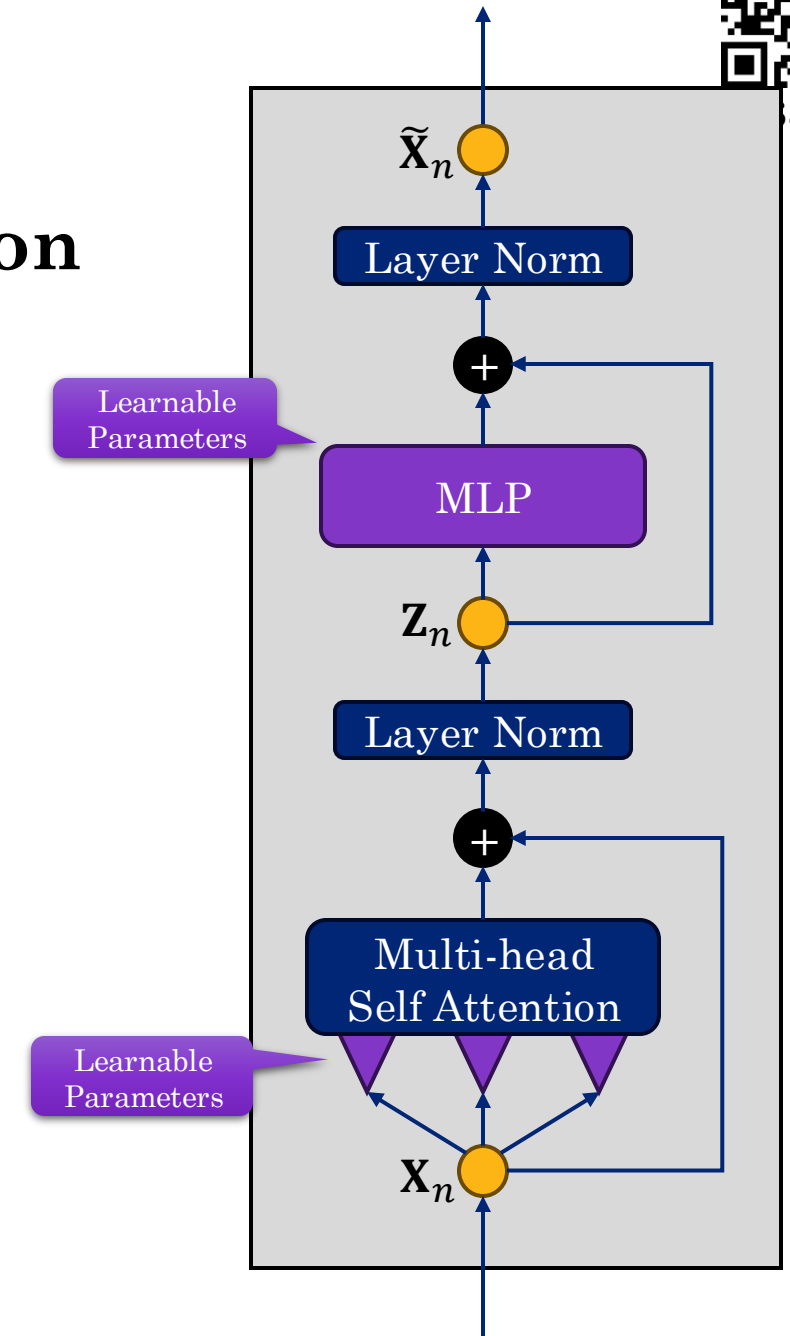
The Transformer Layer

A **transformer layer** combines **self-attention** with a token-level **neural network** (typically, a 2-layer MLP).

- Why is the the MLP is needed?
 - The output of attention is a **linear combination** (αV) of **linear transformations** ($V = XW^{(v)}$) on the input though α is a **non-linear function** of X .

Residual connections and **layer norm** are used to improve training

Transformer layers are repeated **many times** in a transformer model.

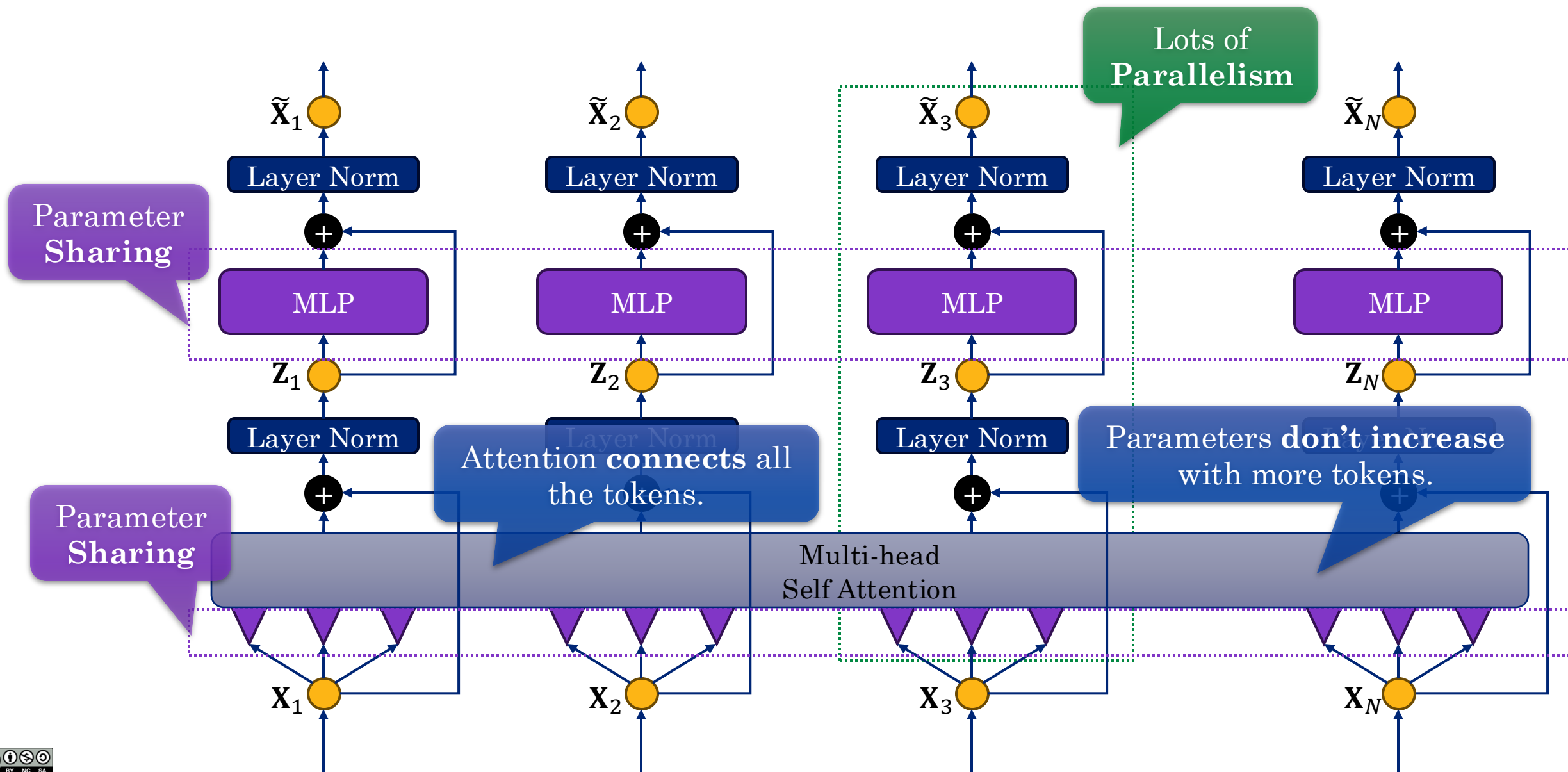


88650

Unrolling the Transformer Layer



9688650

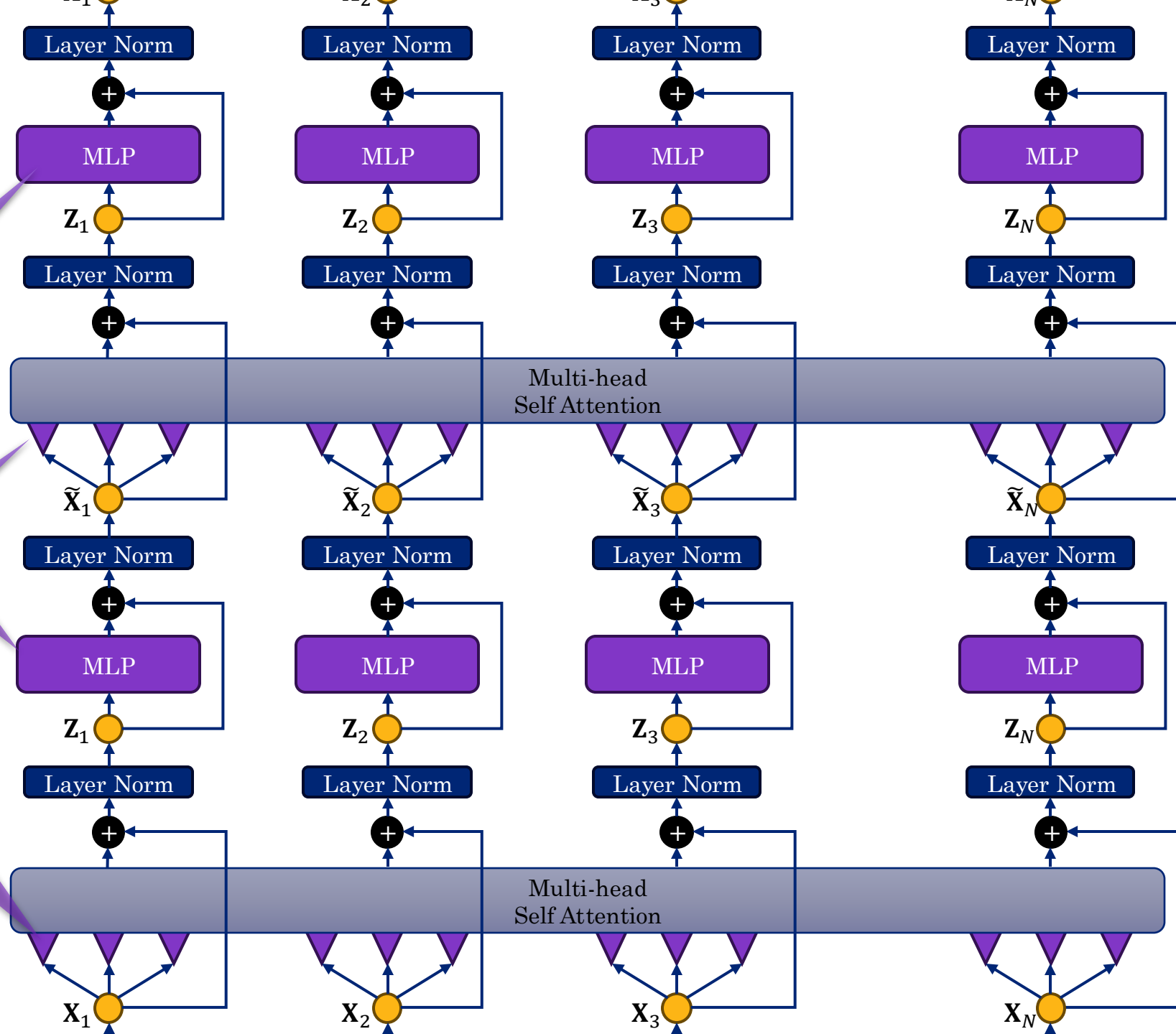


Stacking Transformer Layers

Different Weights!
(not shared)

Different Weights!
(not shared)

Each layer has
new weights.



Reading the Output of the Transformer



Suppose we want to apply the **transformer model** architecture to **image classification**.



Reading the Output of the Transformer



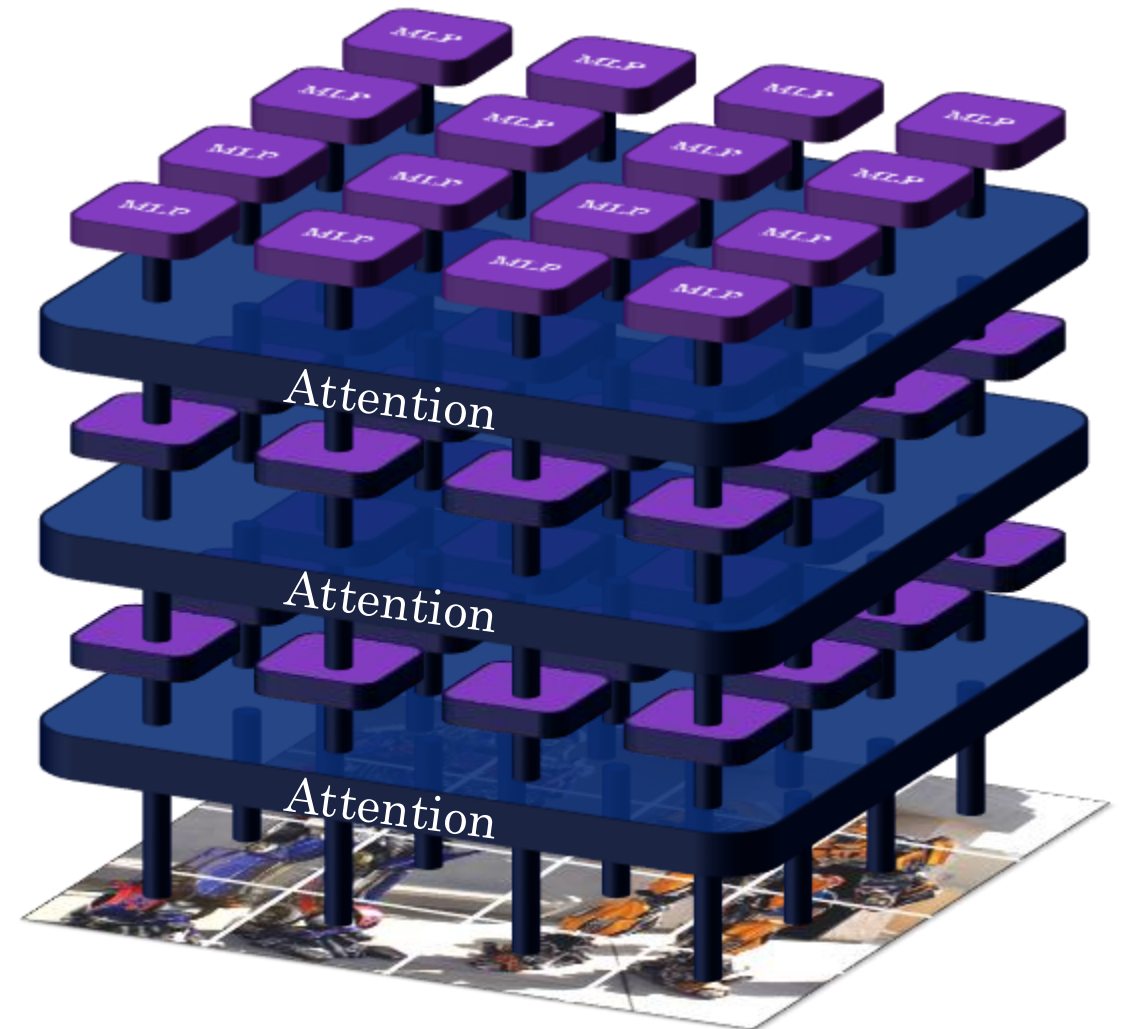
Suppose we want to apply the **transformer model** architecture to **image classification**.

The top layer now has many MLP outputs.

How do we combine them?

- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:



Reading the Output of the Transformer



Suppose we want to apply the **transformer model** architecture to **image classification**.

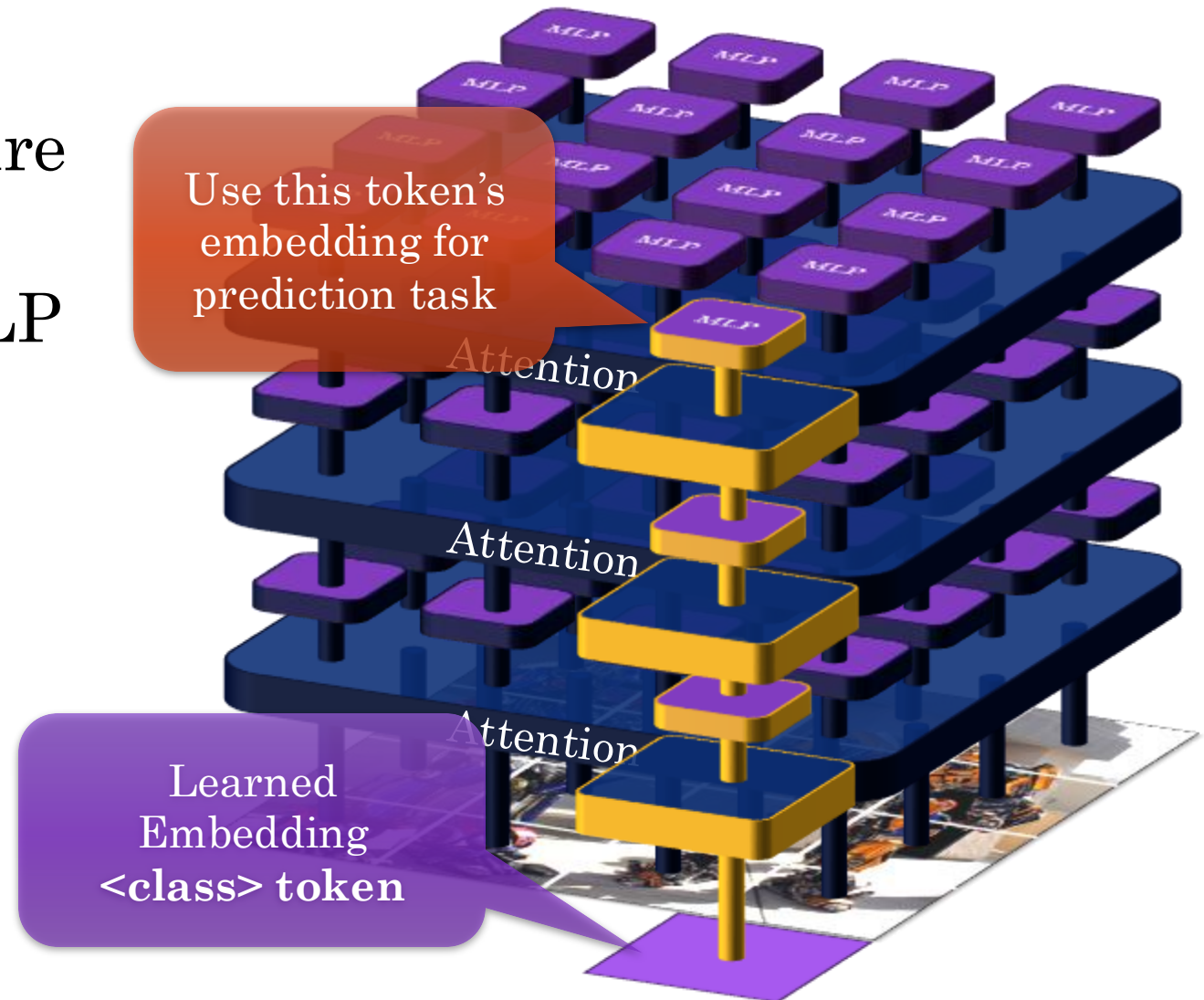
The top layer now has many MLP outputs.

How do we combine them?

- Pooling + Linear Layer
- Concatenation + Layer

Common Trick:

- **Add another token**





Transformers and Token Ordering

Does the **order of tokens matter** in the transformer model?

If I **rearrange the tokens**, would I get a different output for **each token**?



**Does the order of tokens matter
in the transformer model that we
have described so far?**



Transformers and Token Ordering

Does the **order of tokens matter** in the transformer model?

If I **rearrange the tokens**, would I get a different output for **each token**?



Recall the attention operation is:

$$\text{Attention}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = \sum_{i=1}^N \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^N \left(\text{Softmax} \left[\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

the **weighted averaging** which is **order invariant**.

- Attention is **equivariant** – permuting the input tokens permutes the outputs.

Is this a problem?

- **No.** If you want a neural network for **sets of objects**.
- **Yes!** If you want to model **images** or **text**.

Token Position in Images and Text



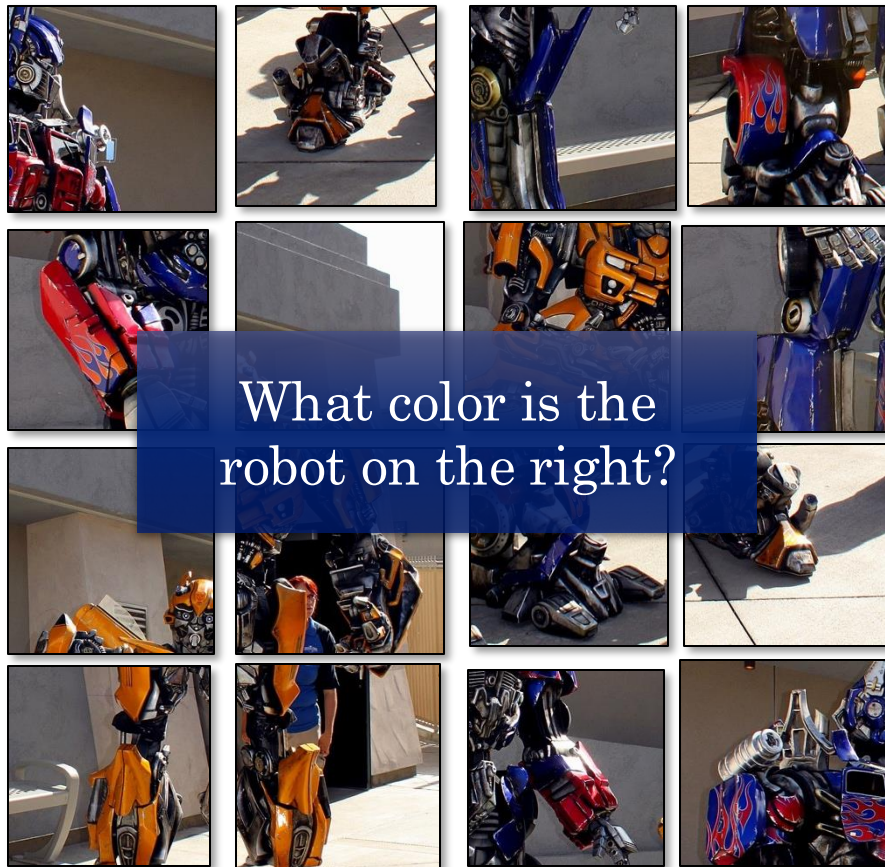
Positional information is important for understanding images and text.



Token Position in Images and Text



Positional information is important for understanding images and text.

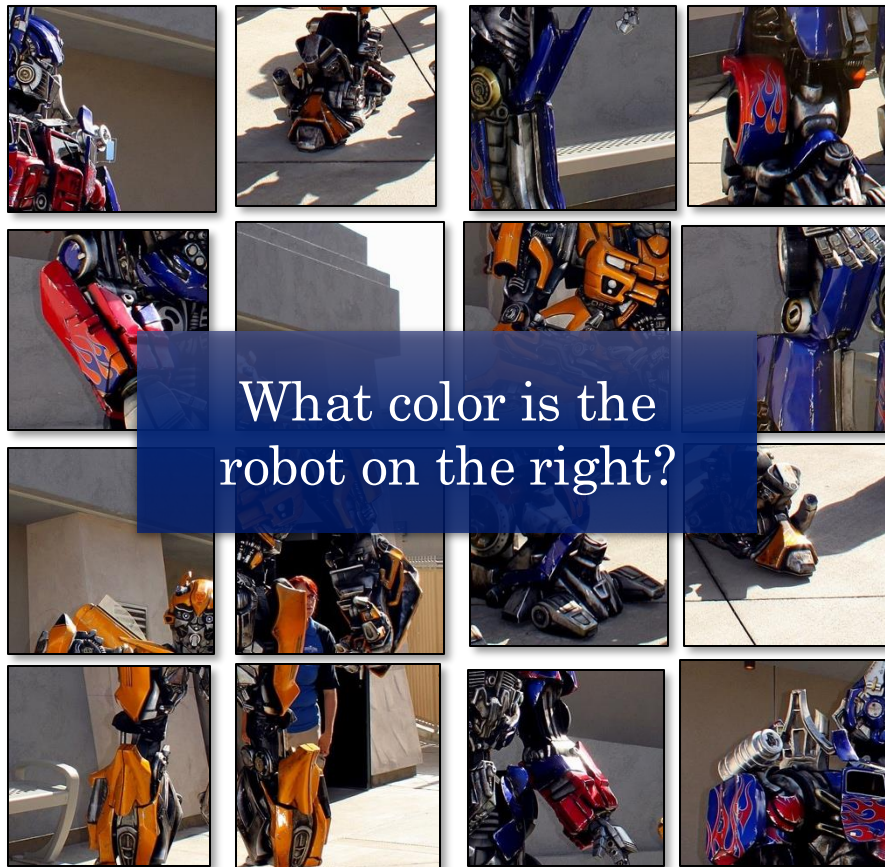


The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

Token Position in Images and Text



Positional information is important for understanding images and text.



The	food	was	good	not	bad	at	all!
-----	------	-----	------	-----	-----	----	------

The	food	was	bad	not	good	at	all!
-----	------	-----	-----	-----	------	----	------

A small permutation in the order changes the meaning!



Positional Encoding

Modify the raw features to contain information about the position of the token in the input. (**Feature Engineering**)

We add a **positional embedding** $\mathbf{r}_n \in \mathbb{R}^D$ **vector** encoding the position to the initial input embedding of each token $\tilde{\mathbf{x}}_n$:

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

- Why not **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$?
 - Increases the dimensionality changing the model.
- Wouldn't this corrupt $\mathbf{x}_n$?
 - For high-dimensional vectors $\mathbf{x}_n$ and $\mathbf{r}_n$ are likely to be almost orthogonal.
 - Linear layers would end up adding $\mathbf{x}_n$ and $\mathbf{r}_n$ if we had used **vstack** $[\mathbf{x}_n, \mathbf{r}_n]$
- How do we construct $\mathbf{r}_n$?



Designing the Position Embedding

An ideal positional encoding should:

1. Provide a **unique representation** $\mathbf{r}_n \in \mathbb{R}^D$ for each location.
2. Be **bounded** on the scale of the original features $\mathbf{x}_n$ (e.g., $[-1,1]$).
 - Needed for **stable training**.
3. Easily express **relative distances** between tokens
 - **Inductive Bias:** relative location is important for images and language.
 - Absolute location can also be important.
4. Should work for **arbitrary number of tokens** N
 - Not all sentence or images are the same size.



Learned Positional Embeddings

The first GPT model (GPT-1) used **learned positional embeddings** in which each N **positional embedding vectors** $\mathbf{r}_n \in \mathbb{R}^D$ are learned during the training process.

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

- The vectors $\mathbf{r}_n$ are randomly initialized (small values)

Strengths:

- Expressive and easy to implement

Weaknesses:

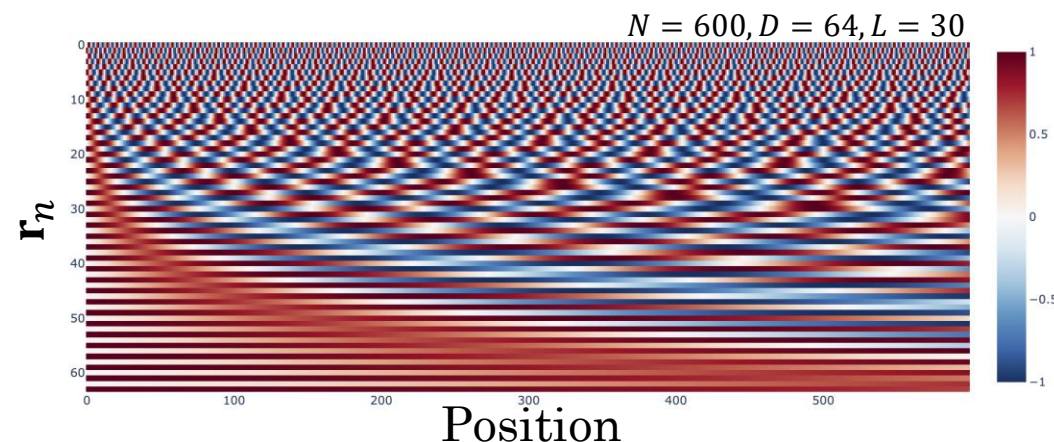
- The maximum N positions needs to be known in advances
- Relative distances can't be easily computed (needs to be learned)

Sinusoidal Positional Encoding



The original Transformer paper used sinusoidal embeddings to encode positions through a combination of waves.

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$



Like a “binary” encoding of the position but continuous:

Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
2	0	0	0	1	1	1	0	0	0	1	1	1	0	0	0	1
3	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
4	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1



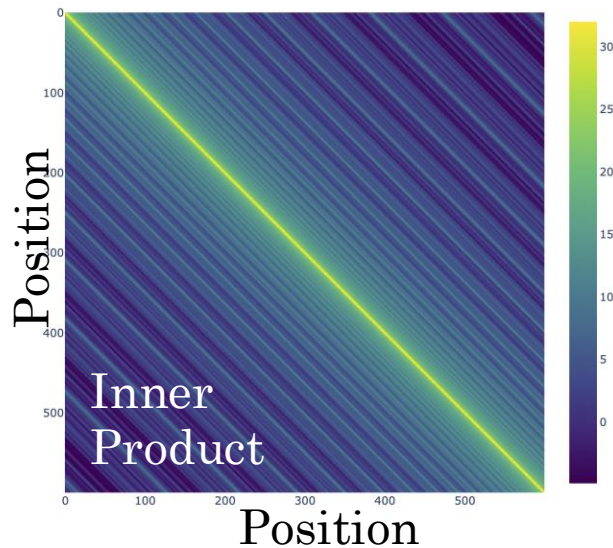
Querying a Relative Position

The sinusoidal position enables **linear layers** to query a fixed offset.

Suppose we have $\mathbf{r}_n$ and I want to query a key at $\mathbf{r}_{n-\Delta}$ then there exists a rotation matrix $\mathbf{R}_\Delta$ such that:

$$\mathbf{r}_{n-\Delta} = \mathbf{R}_\Delta \mathbf{r}_n$$

Inner-products of these encodings decrease with distance:





Rotary Positional Embeddings (RoPE)

Today most models use some variation of **Rotary Positional Embeddings (RoPE)** which “rotate” the **key** and **query vectors** by a **fixed amount** based on the **position**.

- **Consecutive pairs** of dimensions are **rotated at different frequencies**.

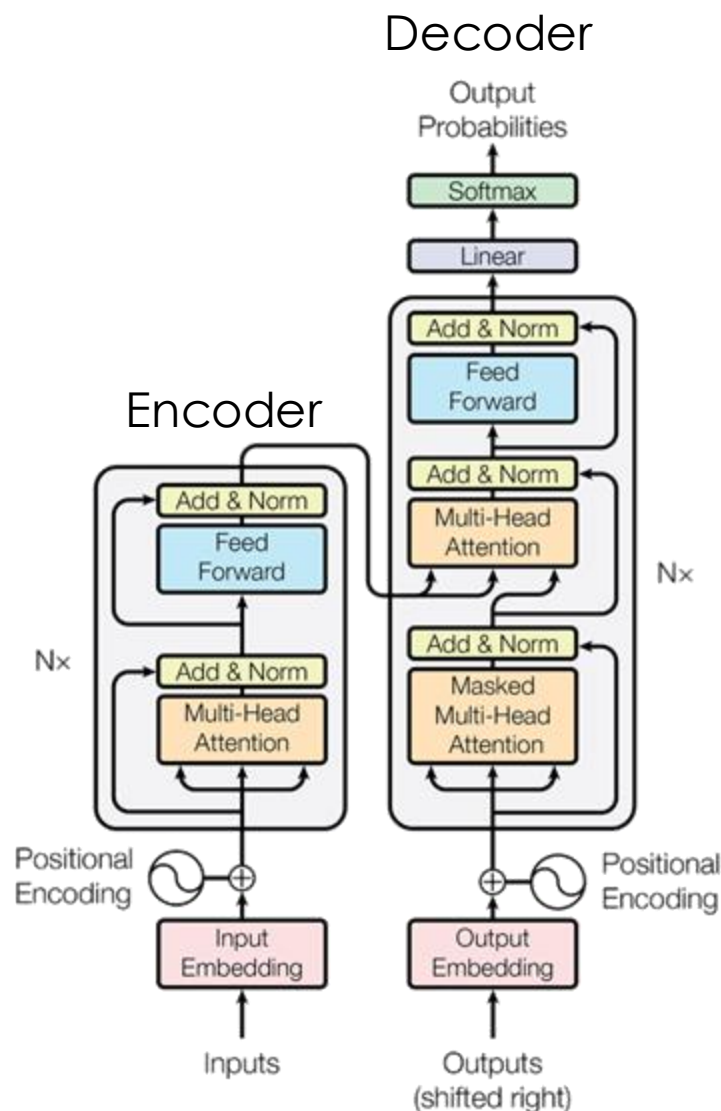
Key features:

- Attention only depends on **relative position** and **relevance**
- Rotation **preserves the norm** of the underlying vector (added transformations do not).
- Can naturally **interpolate positions** between positions

The Encoder Transformer



9688650



arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

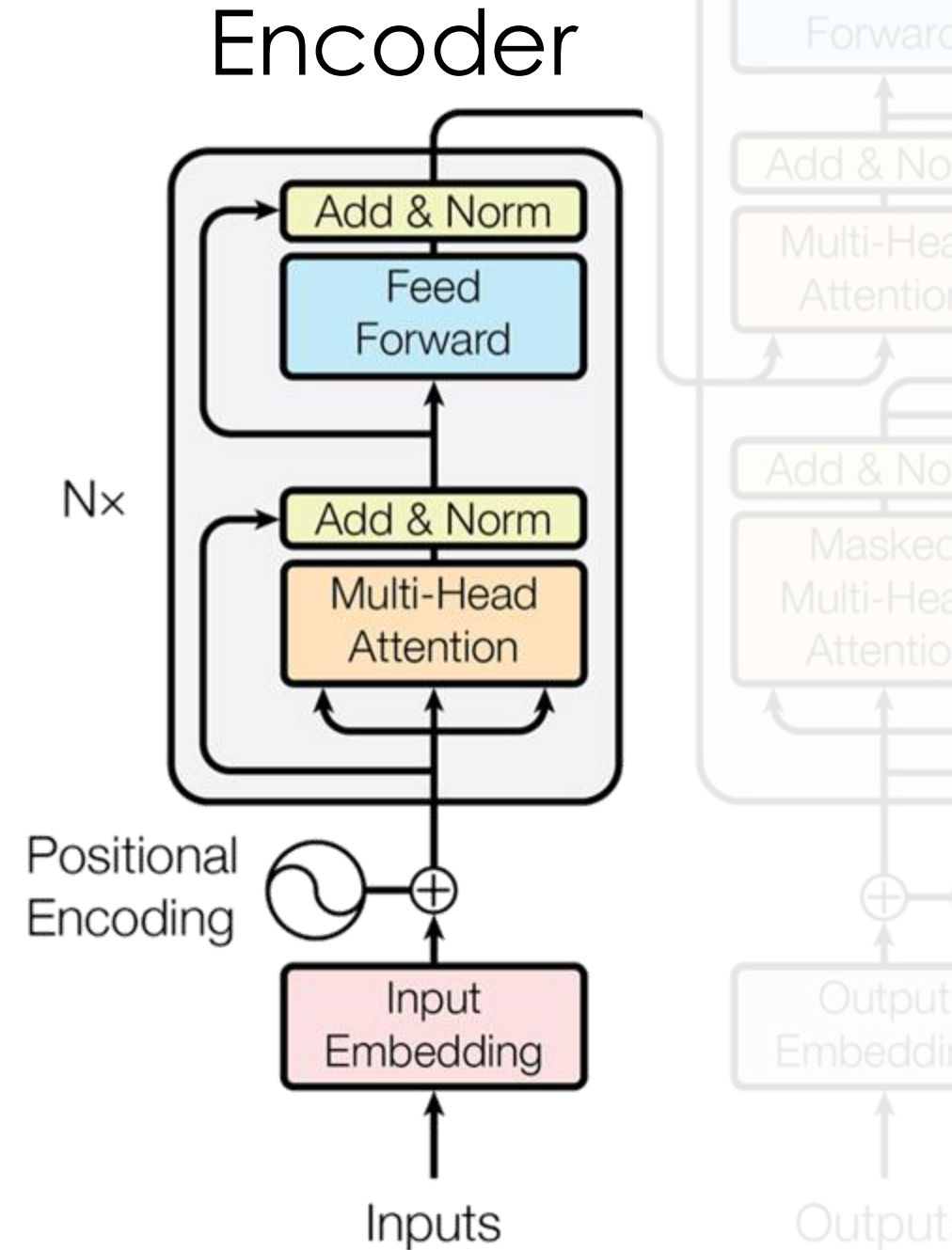
[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

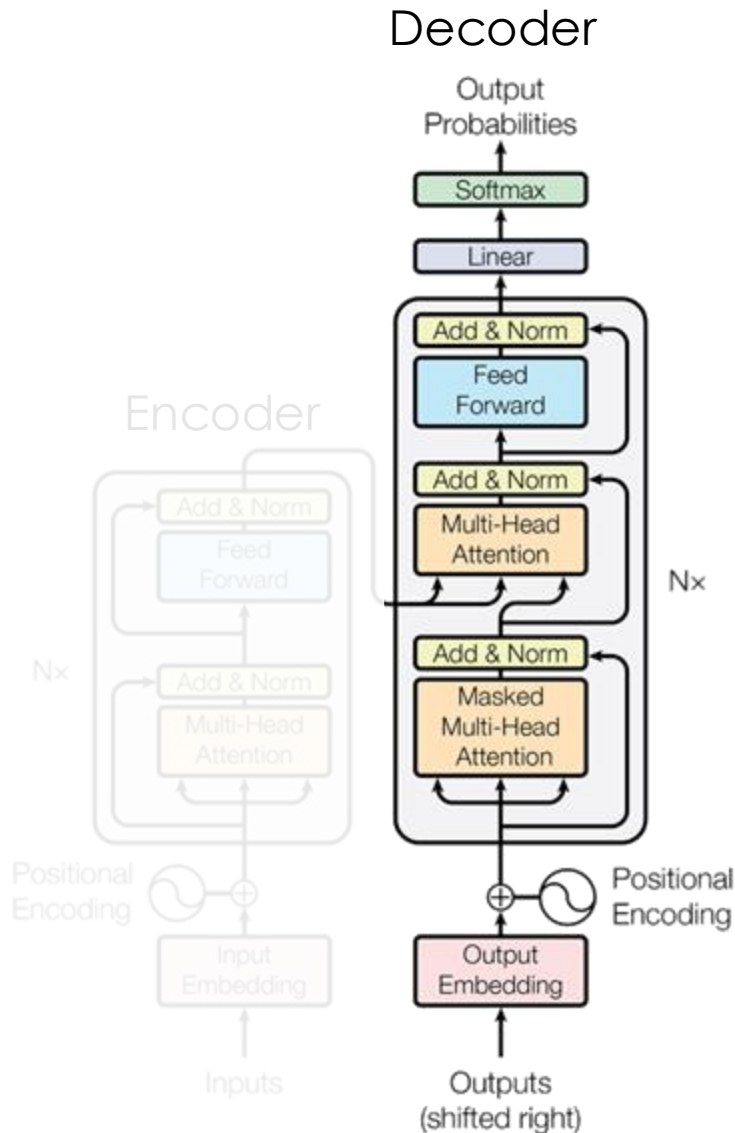
The Encoder Transformer

- Construct input embeddings
 - **Example:** Pixel patches
- Apply Positional Encoding
 - **Example:** Sinusoidal encoding
- Many repeated transformer blocks
- Use final output in downstream tasks
 - **Example:** Pooling or Special token

The encoder transformer architecture is standard for **computer vision tasks** and **language embedding** tasks.



The Decoder Transformer



The Encoder Transformer

- Designed to build a **representation** of the input to a text translation task
 - **Example:** Input sentence in Spanish

The Decoder Transformer

- Designed to **generate** the output text
 - **Ex:** Translation $\text{Enc}(\text{Spanish}) \rightarrow \text{Dec}(\text{English})$

What is the difference?

- **Masked Attention** – today's lecture!

Generative Language Modeling

What is a Large Language Model (LLM)?



- **Large:** The model parameters (**w**) are ***BIG!***
 - ***BILLIONS (now TRILLIONS) of PARAMETERS!!!!***
- **Language Model:** predicting language (e.g., words)

The capital of California is **Sacramento**
San Francisco (1862)
Benicia (1853)
Vallejo (1852)

Tell a short story about a
fairy princess named Alice.

Once

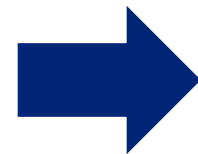


Predicting the Next Word is *Knowledge*

The capital of California is Sacramento
San Francisco (1862)
Benicia (1853)
Vallejo (1852)

Predicting the next word allows you to:

- Answer questions
- Tell stories
- Accomplish tasks

 **Generative AI**

How do we *model* the next ~~word~~ ^{token}?



Modeling **Tokens** not **Words**

Tokens represent **words**, **word parts**, and **special characters**

“The smallest tokenizer!” →

Tokens: [“The”, “ small”, “est”, “ token”, “izer”, “!”]

Constructed based on **frequency of char. sequences**

Allows for **new words**, **misspelling**, and **numbers**

Vocabulary Sizes: Llama-2: 32K → Llama-3: 128K tokens



Byte Pair Encoding

- Initial set of tokens consisting of the entire alphabet:
 - all numbers, special characters, emoji, punctuation,
- Identify pairs of tokens that occur commonly in many documents
 - Create a new token from the concatenation of those tokens.

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Tokenizer Demo

Tiktoken App



9688650

```
from transformers import AutoTokenizer

# Load the Qwen tokenizer
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-14B")
tokenizer.vocab_size
```

```
tokenizer.encode("Hello, how are you?")
```

✓ 0.0s

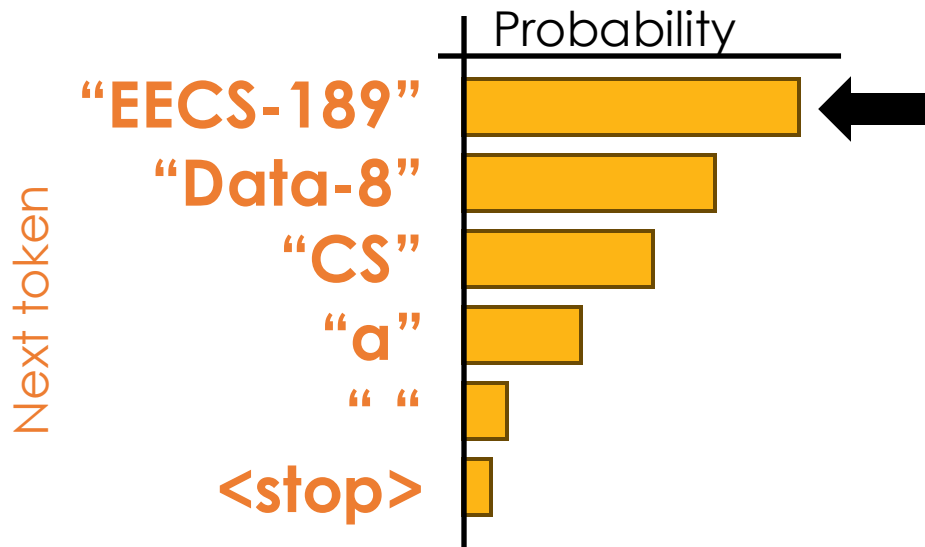
```
[9707, 11, 1246, 525, 498, 30]
```



Causal Language Modeling

The best class at UC Berkeley is _____

Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})$



- Conditioned on the **context**
- Model probability of **next token**
 - **Sample** or **pick most likely**



Causal Language Modeling

The best class at UC Berkeley is _____

Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})$

- Conditioned on the **context**
- Model probability of **next token**
 - Sample or pick most likely

How do we go from
predicting a single token to
writing an essay?

One token at a time!



Basic Auto Regressive Decoding

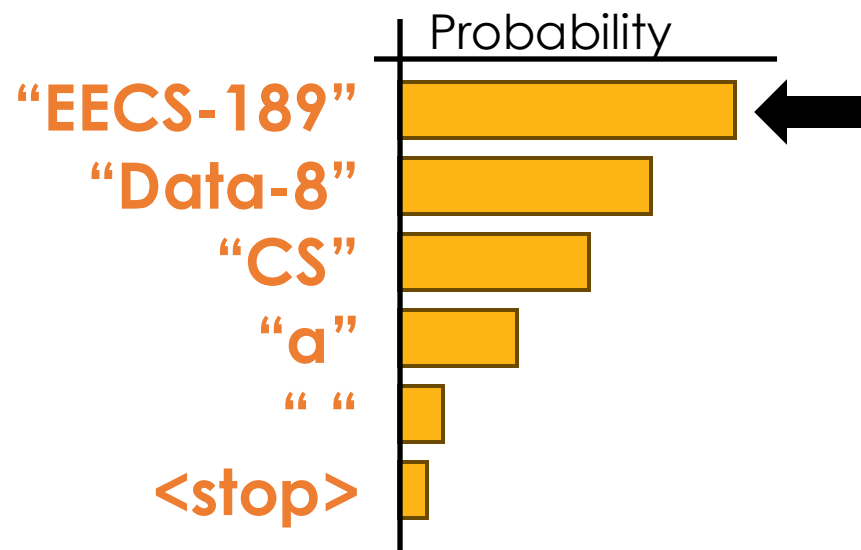
1. **Compute the probability** over the next token
2. **Select the next token**
 1. **Most likely** next token (temperature 0)
 2. **Sample** over the top few most likely tokens
3. **Append the selected token** to the context
4. **Repeat until the <stop> token** is reached.

Auto-regressive Decoding



- Sample **one token at a time** and add to context

Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})$



Decode one word:

The best class at UC Berkeley
is **EECS-189**



Auto-regressive Decoding



- Sample one token at a time and **add to context**

Model: $\Pr(\underbrace{\quad}_{\text{Next Token}} \mid \underbrace{\text{“The best class at UC Berkeley is EECS-189”}}_{\text{Context (ordered tokens)}})])$



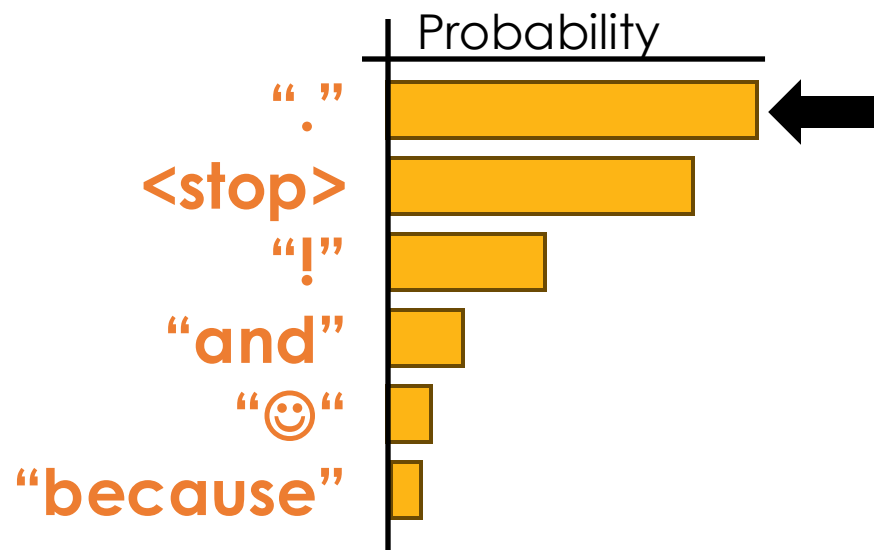
The diagram illustrates the auto-regressive decoding process. It shows a probability distribution $\Pr(\cdot \mid \cdot)$ where the first argument is the 'Next Token' (indicated by an orange bracket) and the second argument is the 'Context (ordered tokens)' (indicated by a blue bracket). The context is the string "The best class at UC Berkeley is EECS-189". A blue arrow points from the text 'add to context' in the list above to the token 'EECS-189' in the context string.

Auto-regressive Decoding



- Sample one token at a time and **add to context**

Model: $\Pr(\underbrace{\quad}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is EECS-189"}]}_{\text{Context (ordered tokens)}})$



Decode one word:

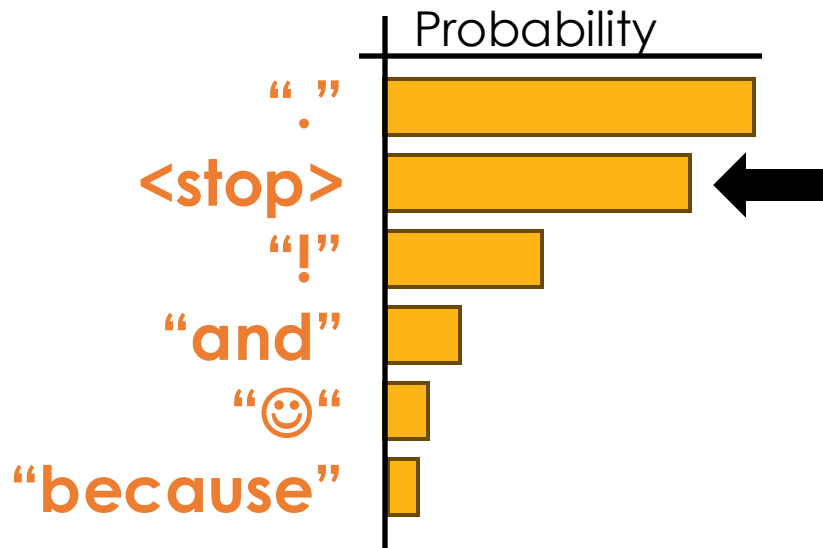
The best class at UC Berkeley
is EECS-189.

Auto-regressive Decoding



- Sample one token at a time and **add to context**

Model: $\Pr(\underbrace{\quad}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is EECS-189"}]}_{\text{Context (ordered tokens)}})$



Sample according to probabilities

Decode one word:

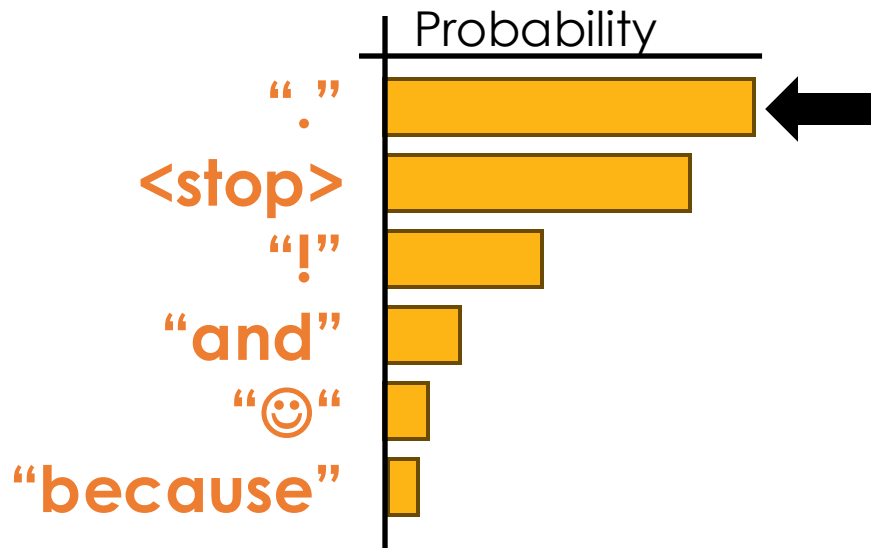
The best class at UC Berkeley
is EECS-189<stop>

Auto-regressive Decoding



- Sample **one token at a time** and add to context

Model: $\Pr(\underbrace{"."}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is EECS-189"}_{\text{Context (ordered tokens)}}])$



Decode one word:

The best class at UC Berkeley
is EECS-189.



Auto-regressive Decoding

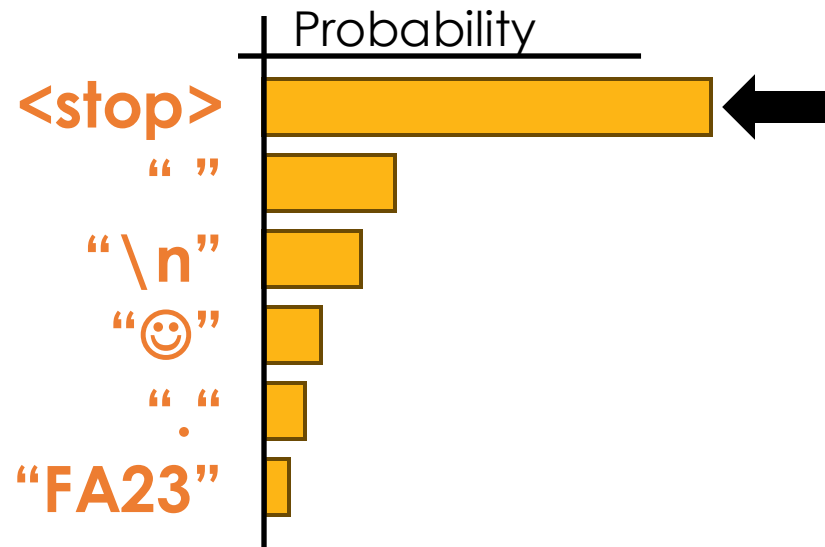


- Sample **one token at a time** and add to context

Model: $\Pr(\text{<stop>} \mid \text{"The best class at UC Berkeley is EECS-189."})$

Next Token (under <stop>)

Context (ordered tokens) (under the string)



Decode one word:

The best class at UC Berkeley
is EECS-189.<stop>

↑ **Stop decoding!**

Quick Recap



- **Causal language modeling** predict next token given context

Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})$

- **Auto-regressive** (iterative) decoding:

The best class at UC Berkeley **is**

The best class at UC Berkeley is **EECS-189**

The best class at UC Berkeley is EECS-189!

The best class at UC Berkeley is EECS-189!<**stop**>

Call model many times.
Slow to compute!

Building a basic Language Model



Model: $\Pr(\underbrace{\text{"EECS-189"}}_{\text{Next Token}} \mid \underbrace{\text{"The best class at UC Berkeley is"}}_{\text{Context (ordered tokens)}})]$

We model this using a **decoder transformer** over **token embeddings**.

Building a basic Language Model



$\text{Pr}(\text{"time"} \mid \text{"Once upon a"})$

Modeling Goal:

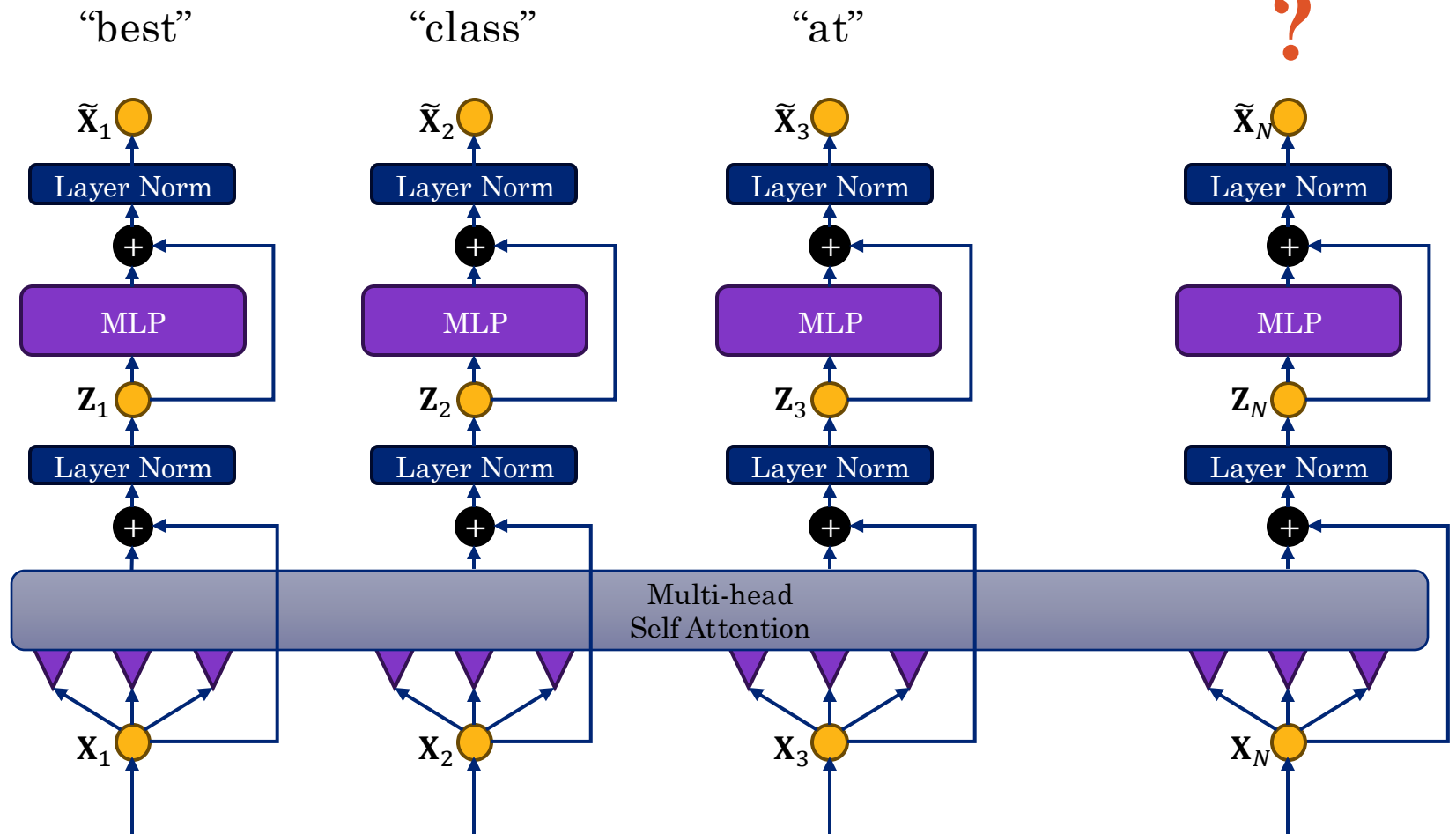
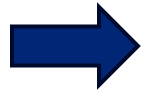
Predict the **next token** (*prob. of all possible next tokens*)
given the context (*all the previous tokens*).

Next token prediction



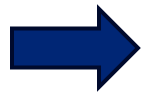
9688650

Output is the
next token



Do you see a
problem with
this setup?

Input is all the
pervious tokens



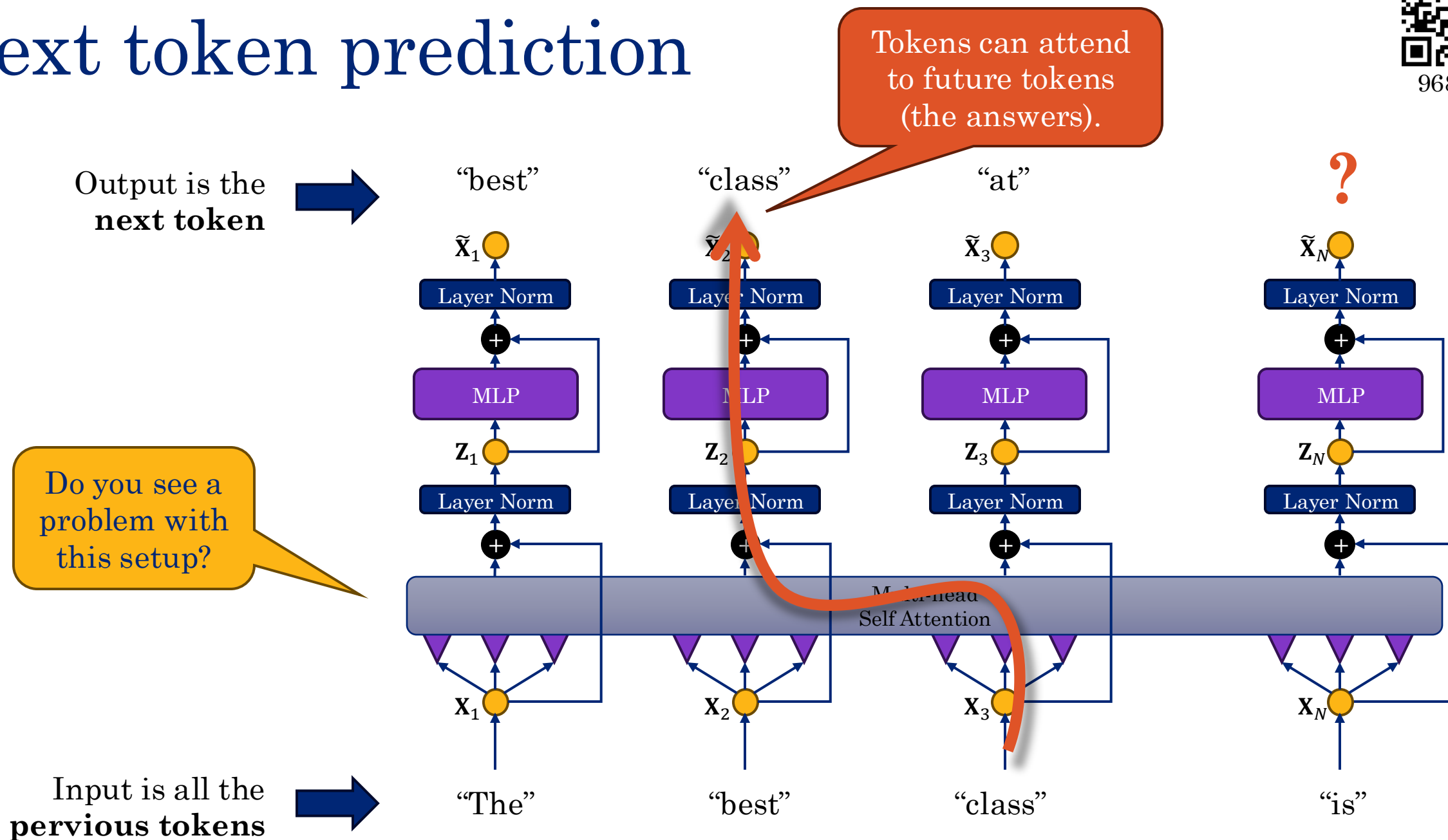


Why is the encoder architecture unsuitable for next-token prediction?

Next token prediction



9688650



The Masked Attention

$$\text{Attn}[\mathbf{K}, \mathbf{Q}, \mathbf{V}]_n = (\boldsymbol{\alpha} \mathbf{V})_n = \sum_{i=1}^n \alpha_{ni} \mathbf{V}_i = \sum_{i=1}^n \left(\text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right] \right)_{ni} \mathbf{V}_i$$

Symbols:

$$\mathbf{V} = \mathbf{X} \mathbf{W}^{(v)}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}^{(k)}$$

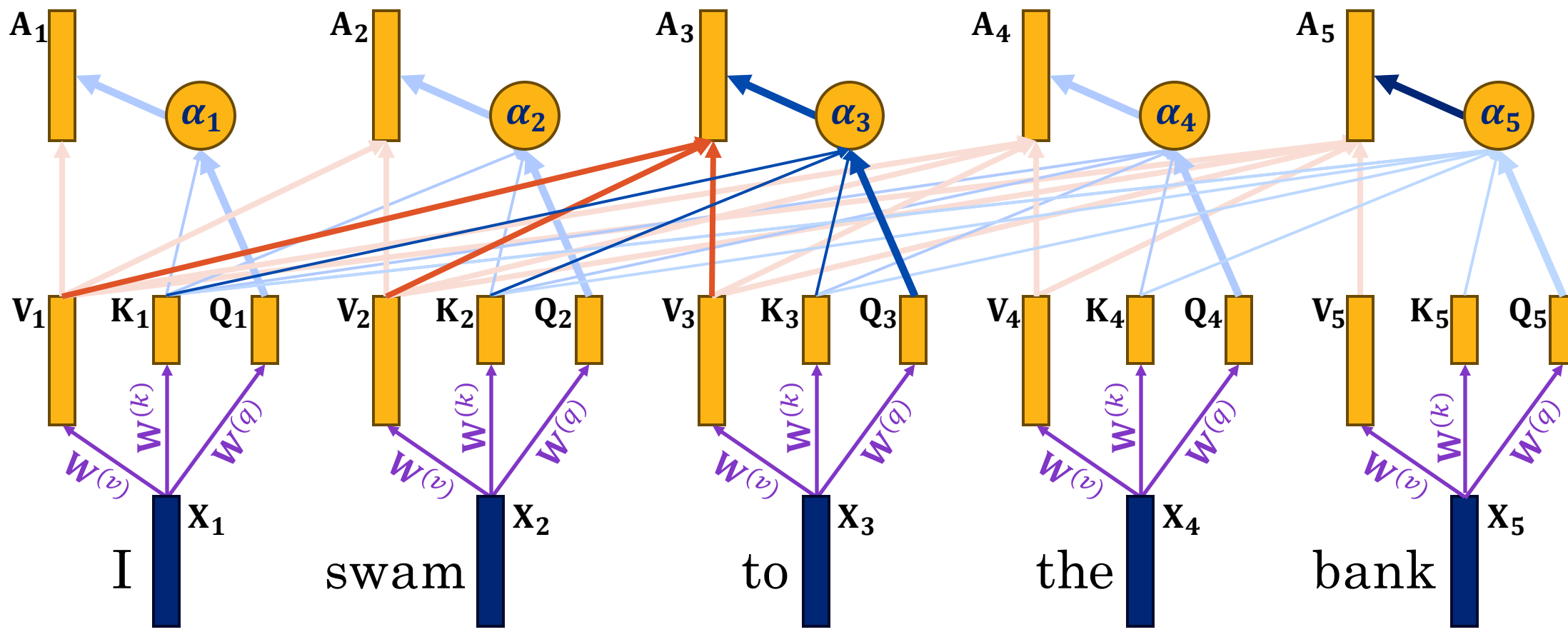
$$\mathbf{Q} = \mathbf{X} \mathbf{W}^{(q)}$$

$$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}} \left[\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{D_k}} \right]$$

$$\boldsymbol{\alpha} \in \mathbb{R}^{N \times N}$$

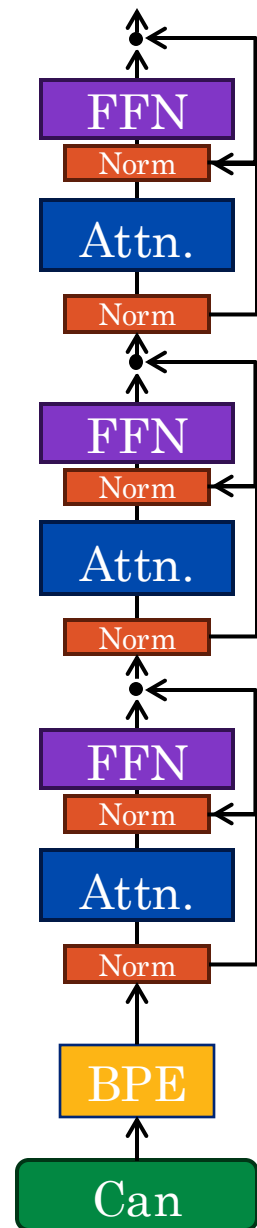


9688650



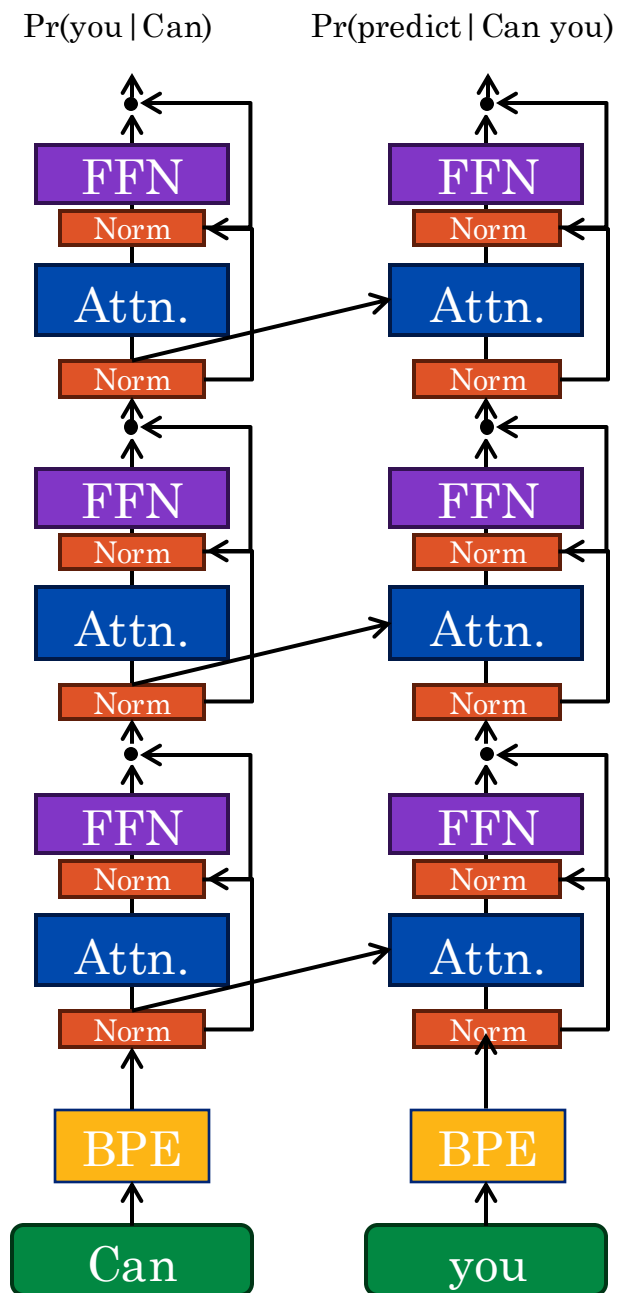
Unrolling the Model

$\Pr(\text{you} \mid \text{Can})$



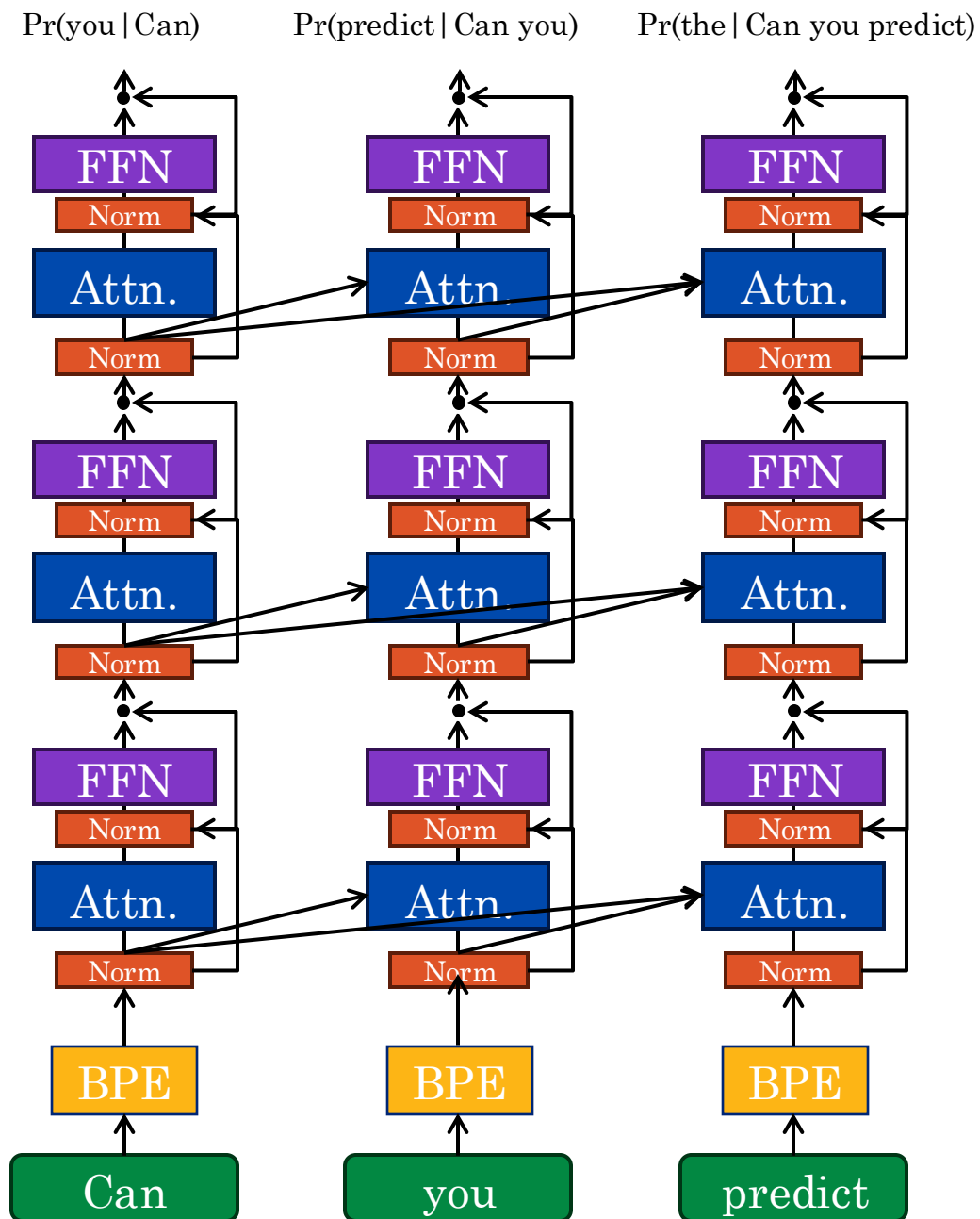
9688650

Unrolling the Model



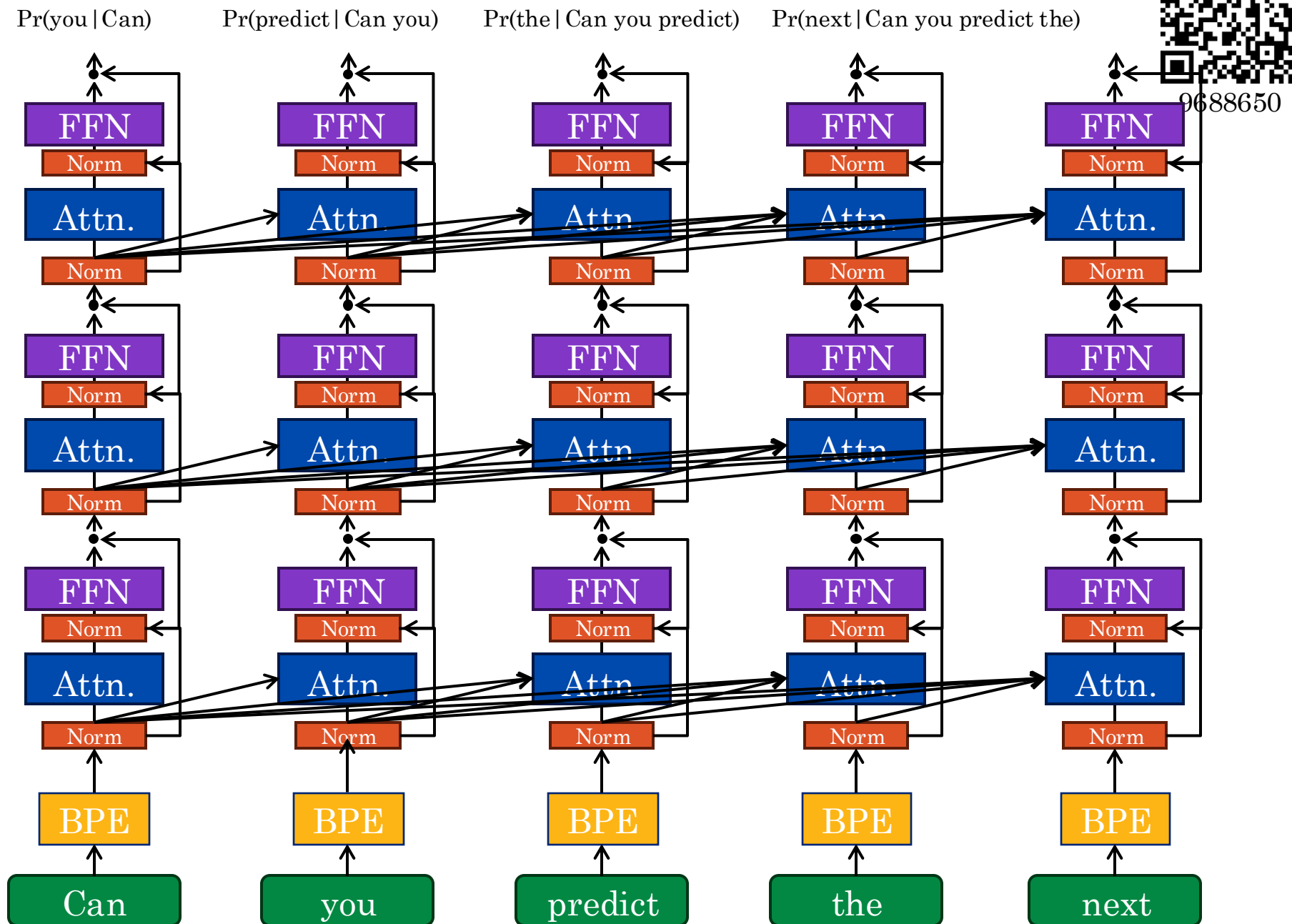
9688650

Unrolling the Model



9688650

Unrolling the Model

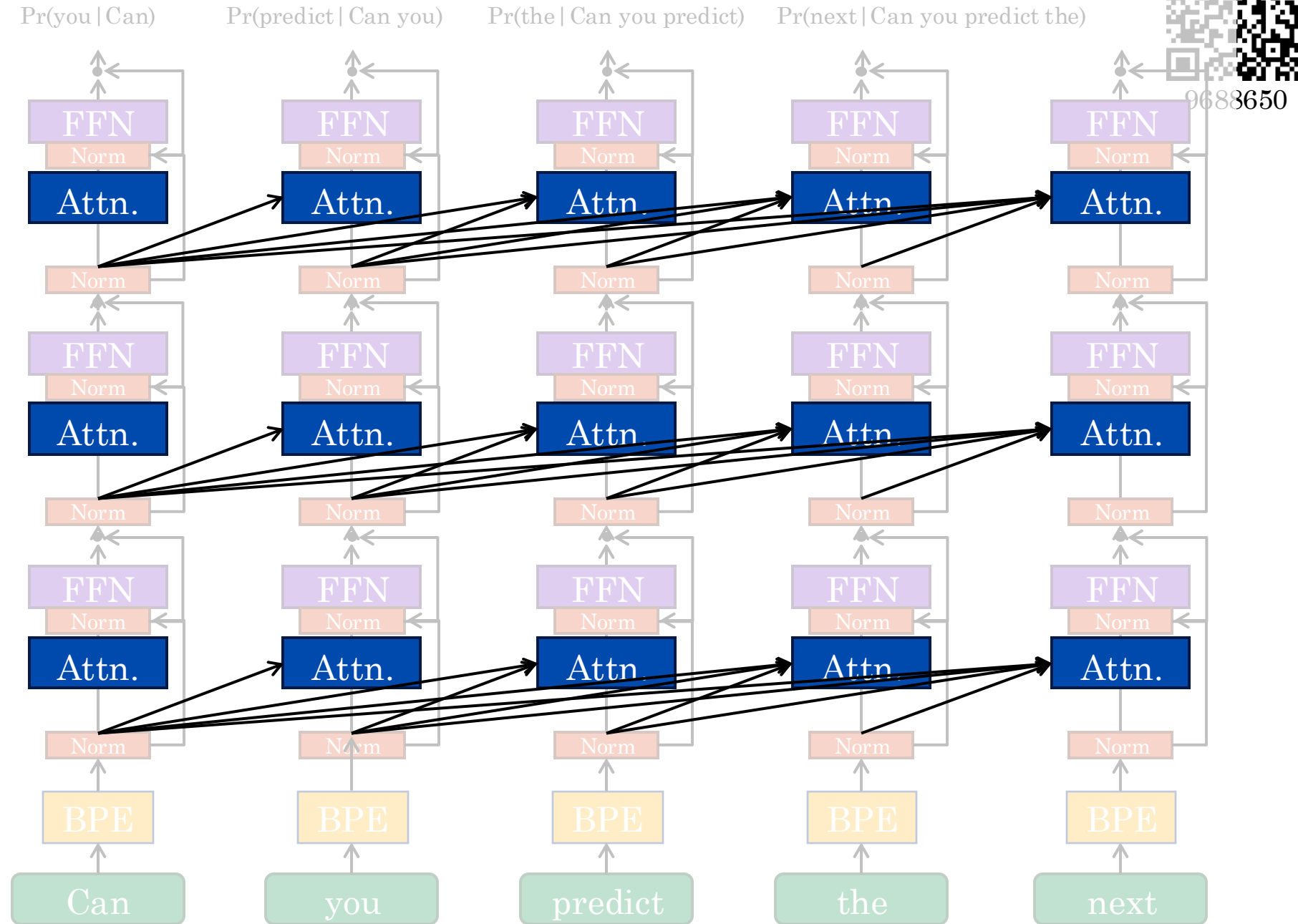




Which token is the most expensive to compute?

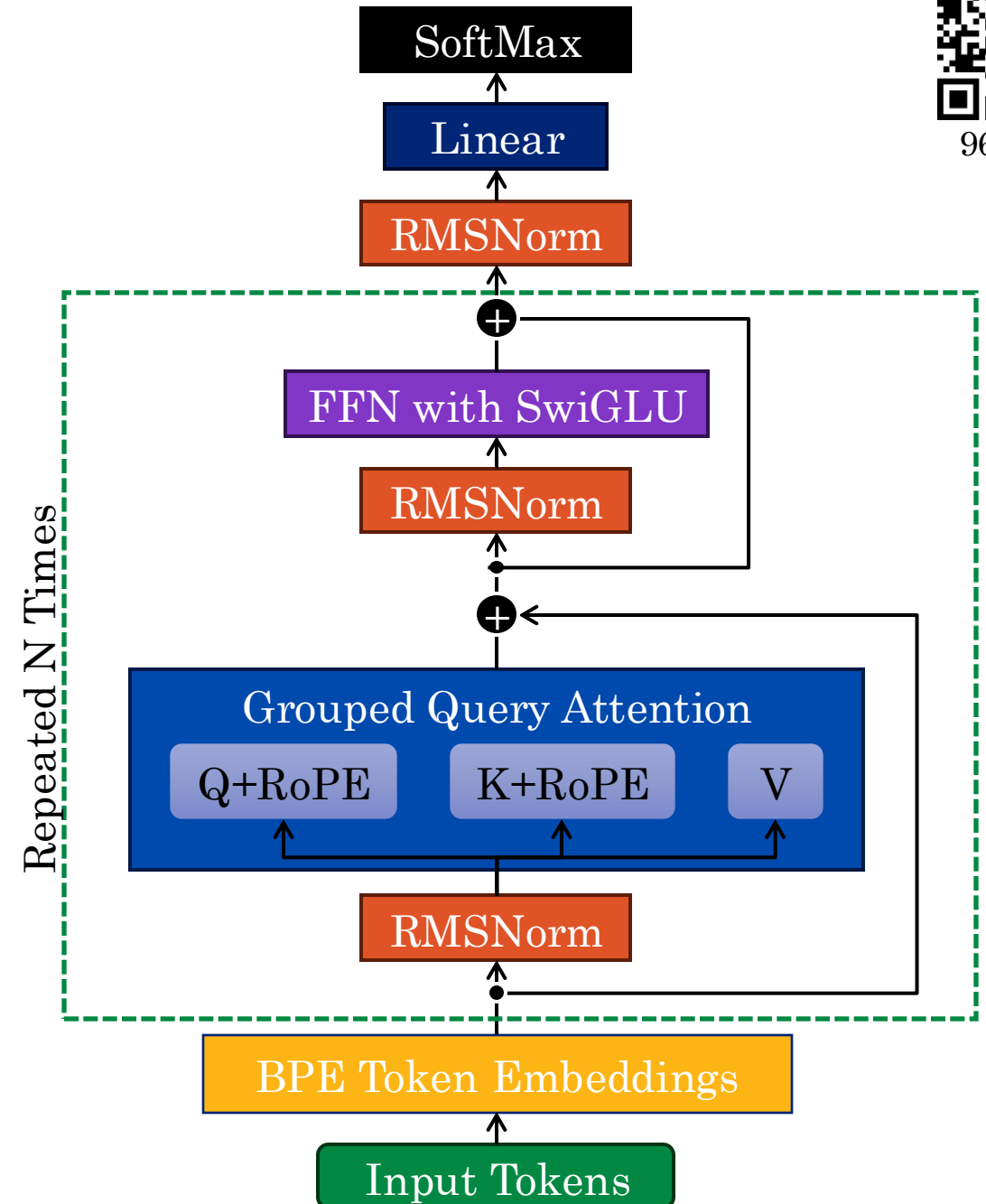
Masked Attention

Auto-regressive
Attend only to
previous tokens
Decoder only
Transformer



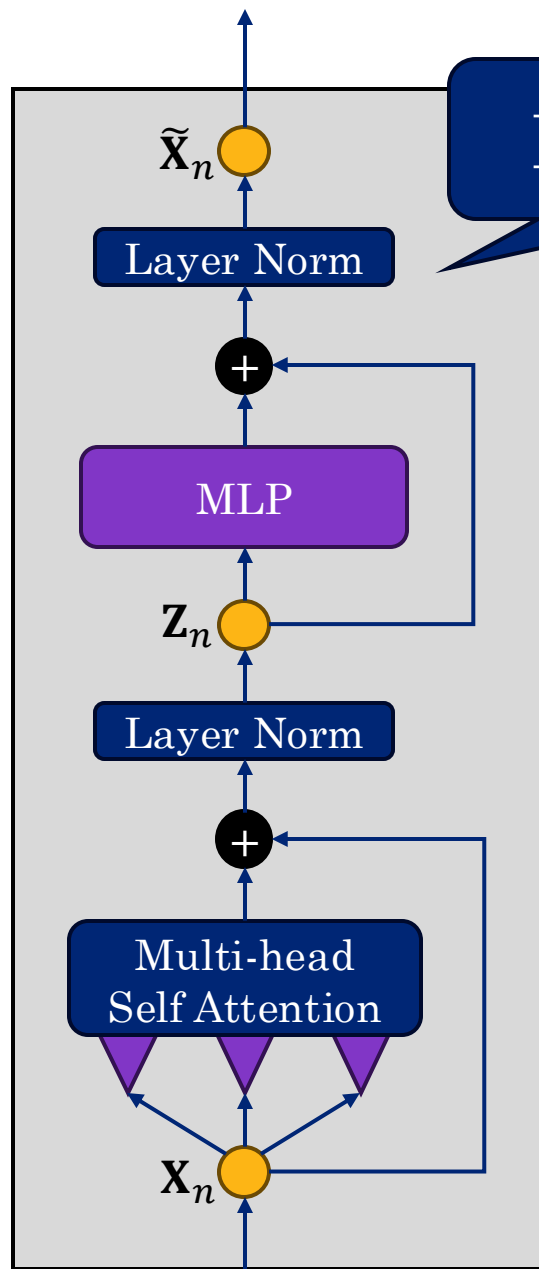
Llama-3 Architecture

- **RMSNorm** (Layer Norm.) – improve training stability
- **FFN with SwiGLU** – Feed forward network
- **Residual Connections** – improve training stability
- **Repeated N Times** – increase model size



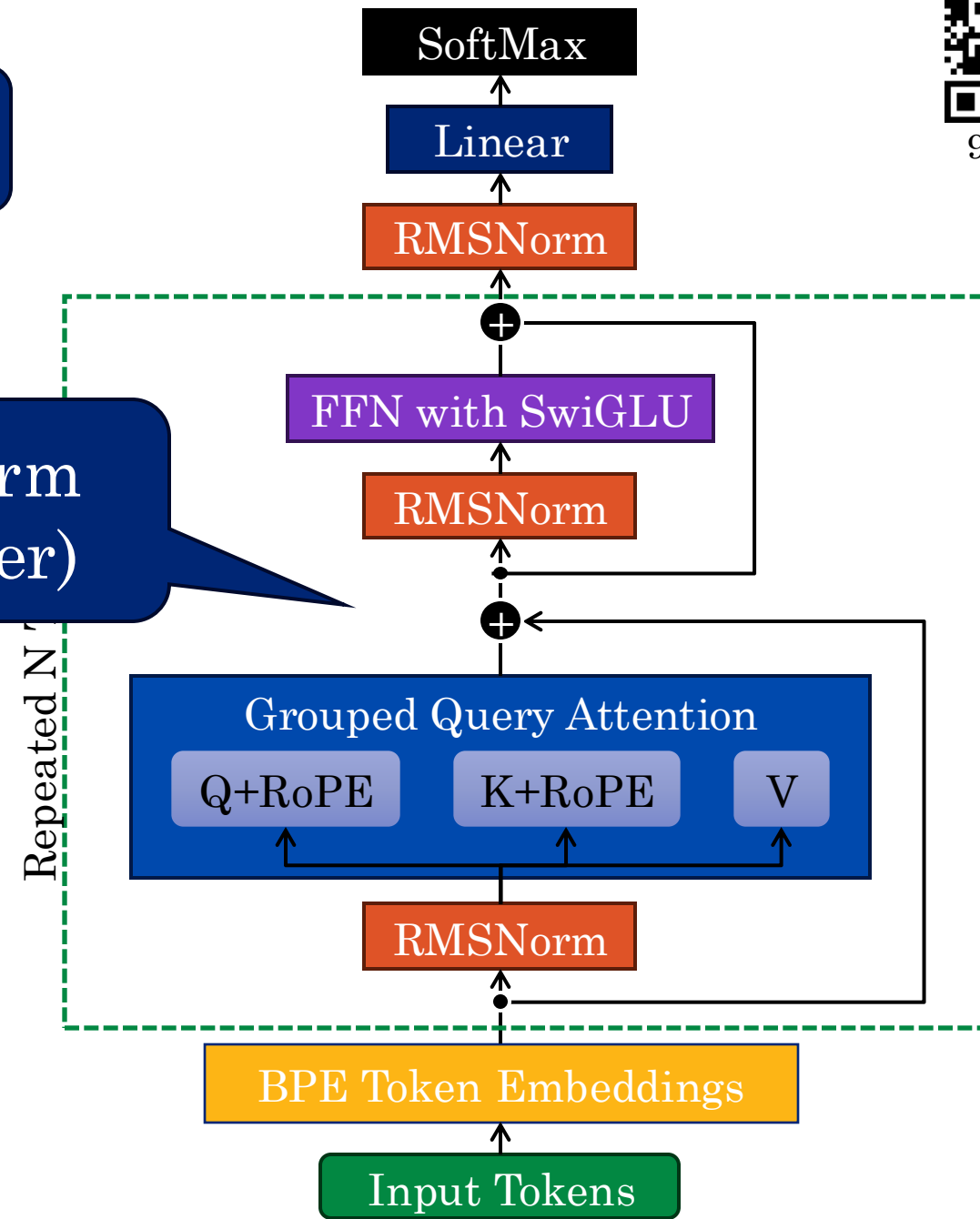
See [Actual Code](#) (its just one python script!)





Post-Norm

Pre-Norm
(is better)



9688650

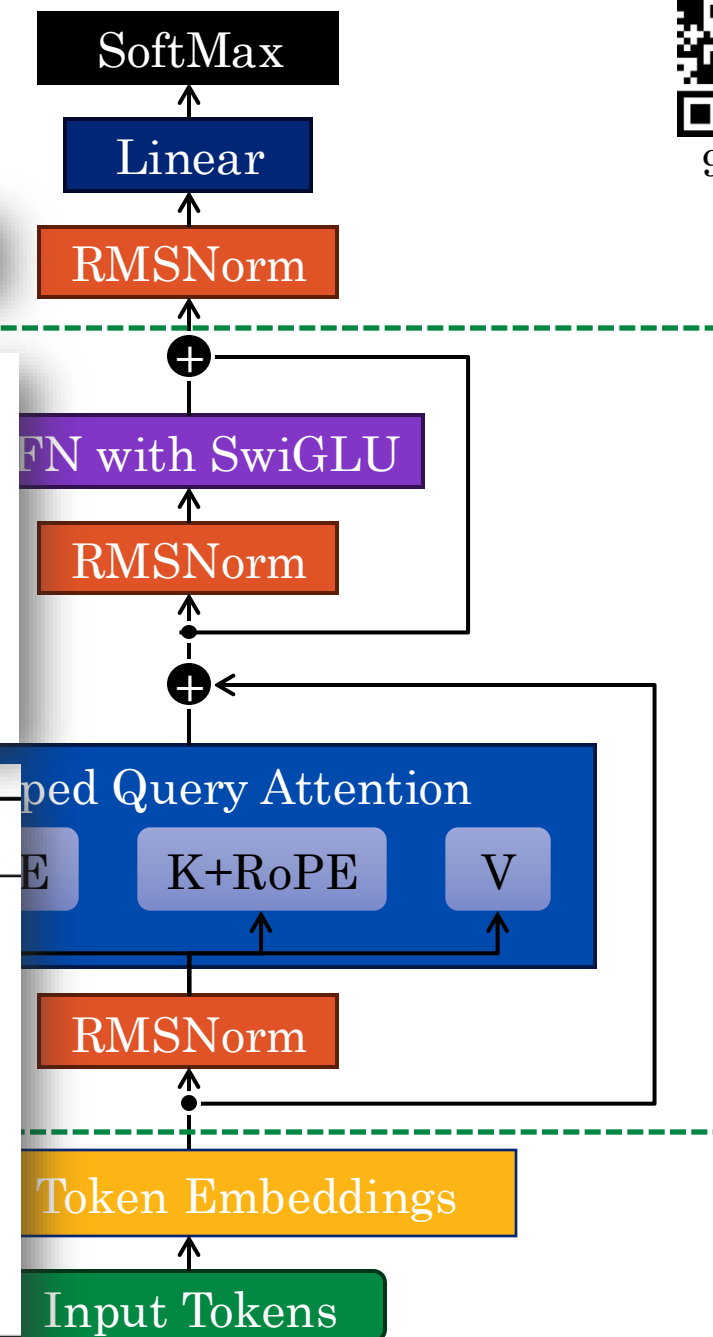
Going Big!

Llama-3 70b Instruct: 8192 hidden size, 80 layers, 64 query heads, 8 kv heads.

Llama 2

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

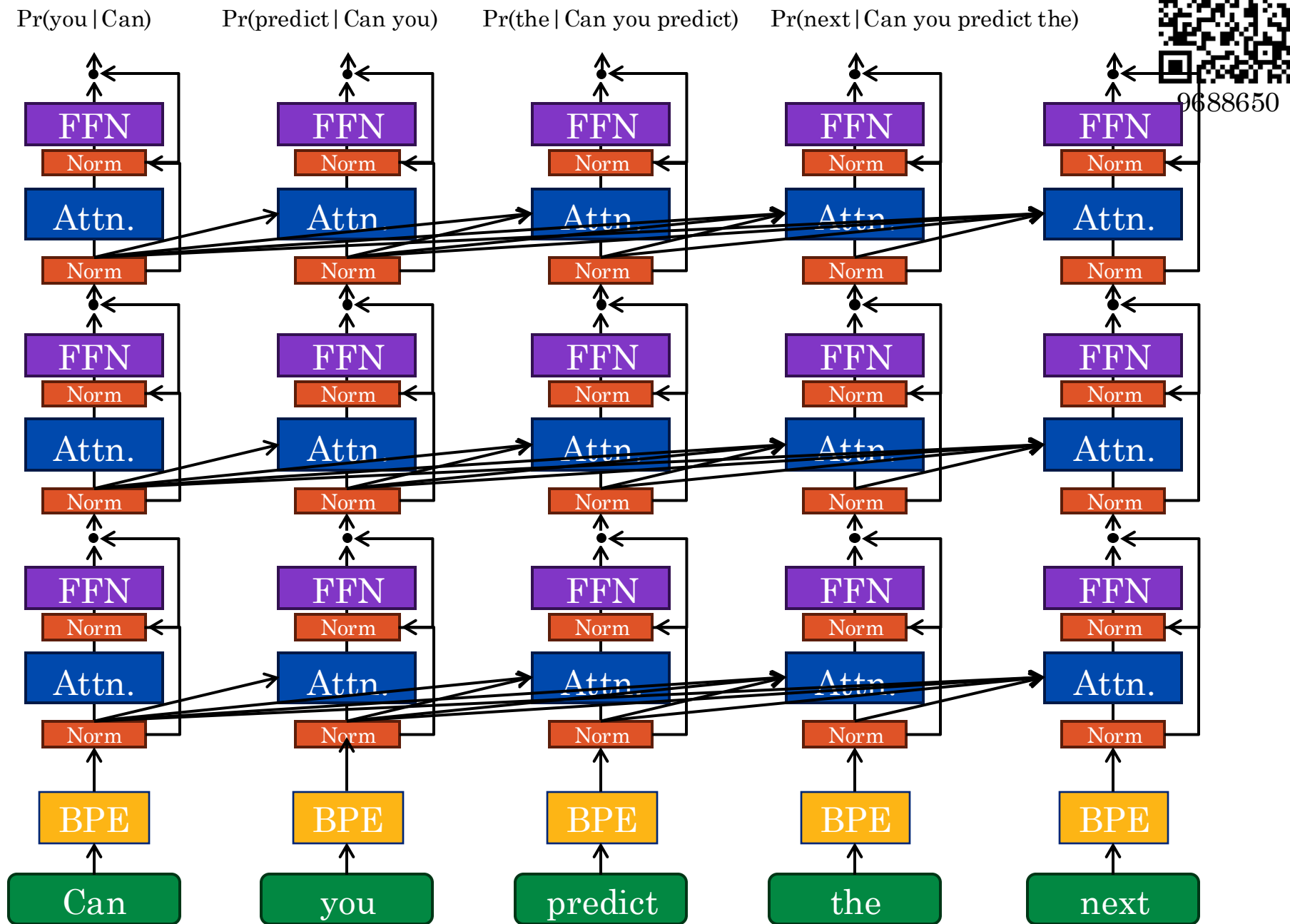
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128



9688650

Question

Which token requires the most computation?



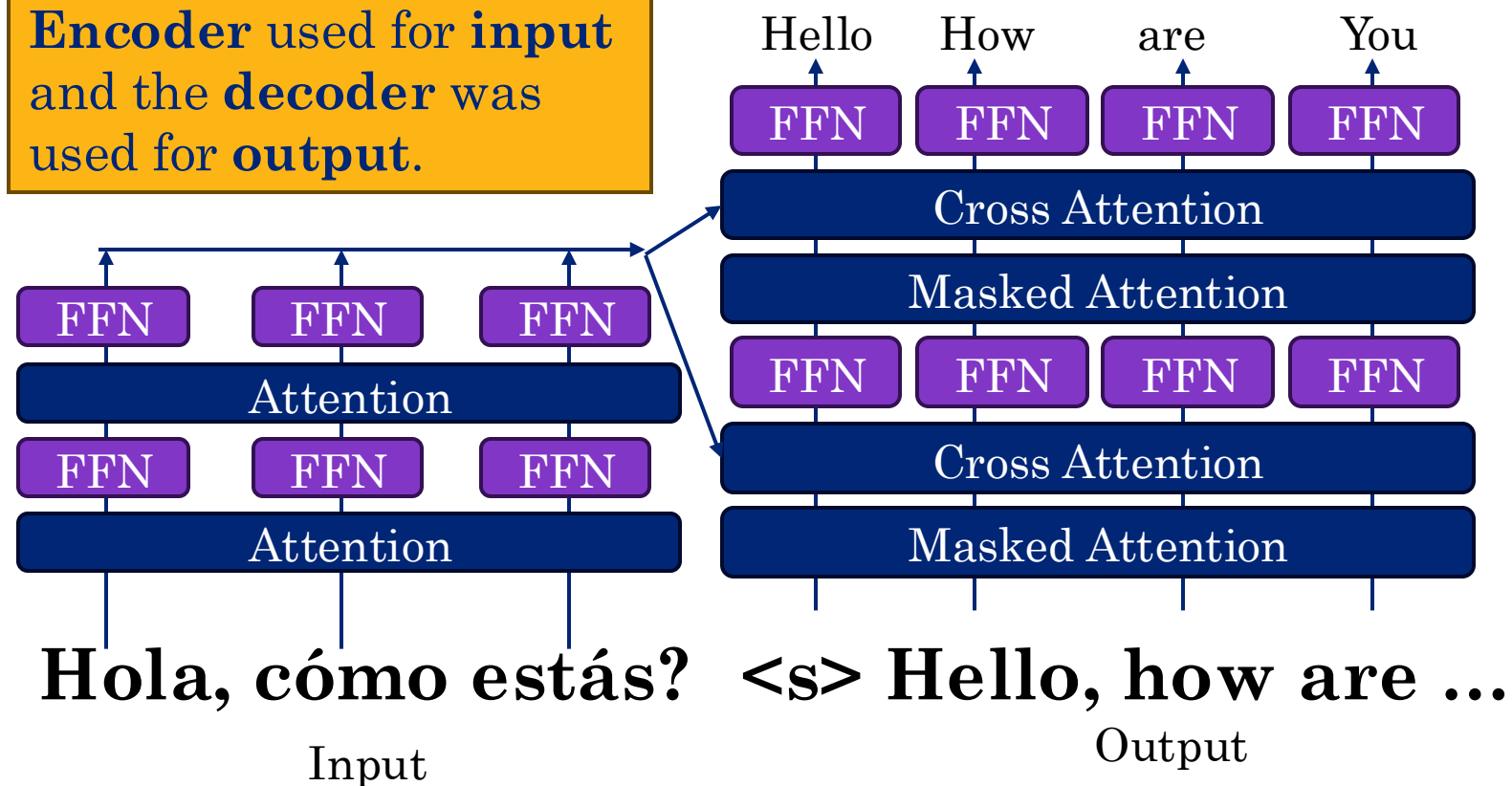
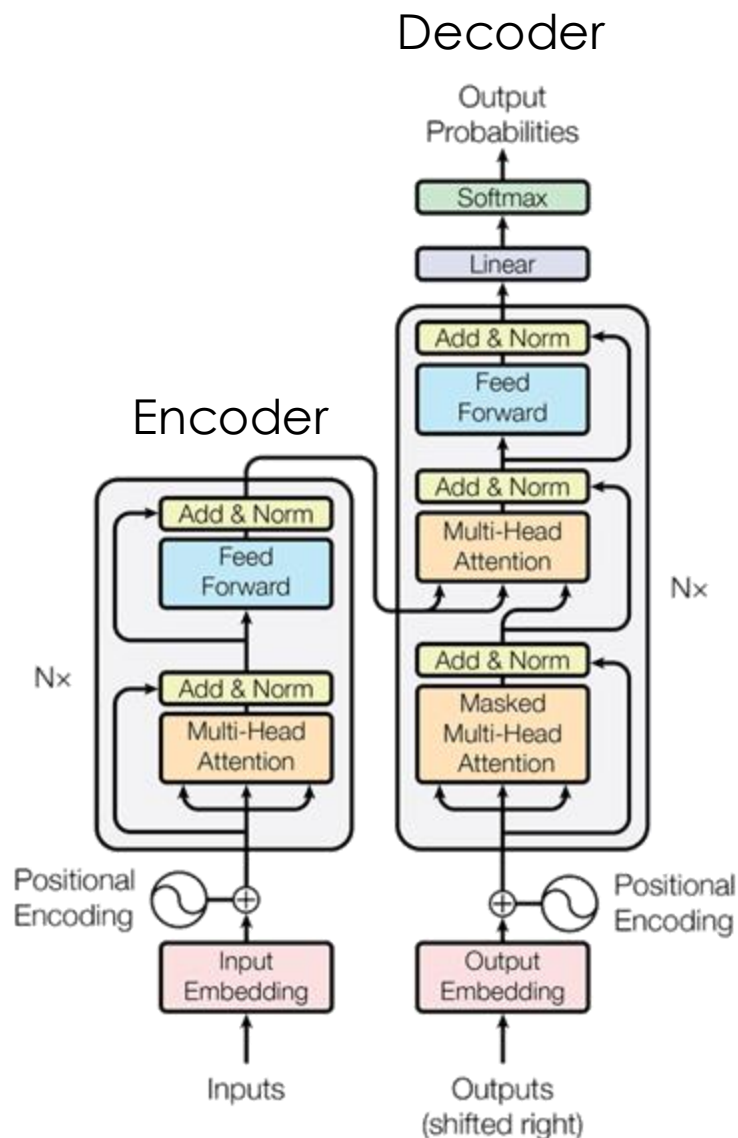
The Encoder-Decoder Transformer



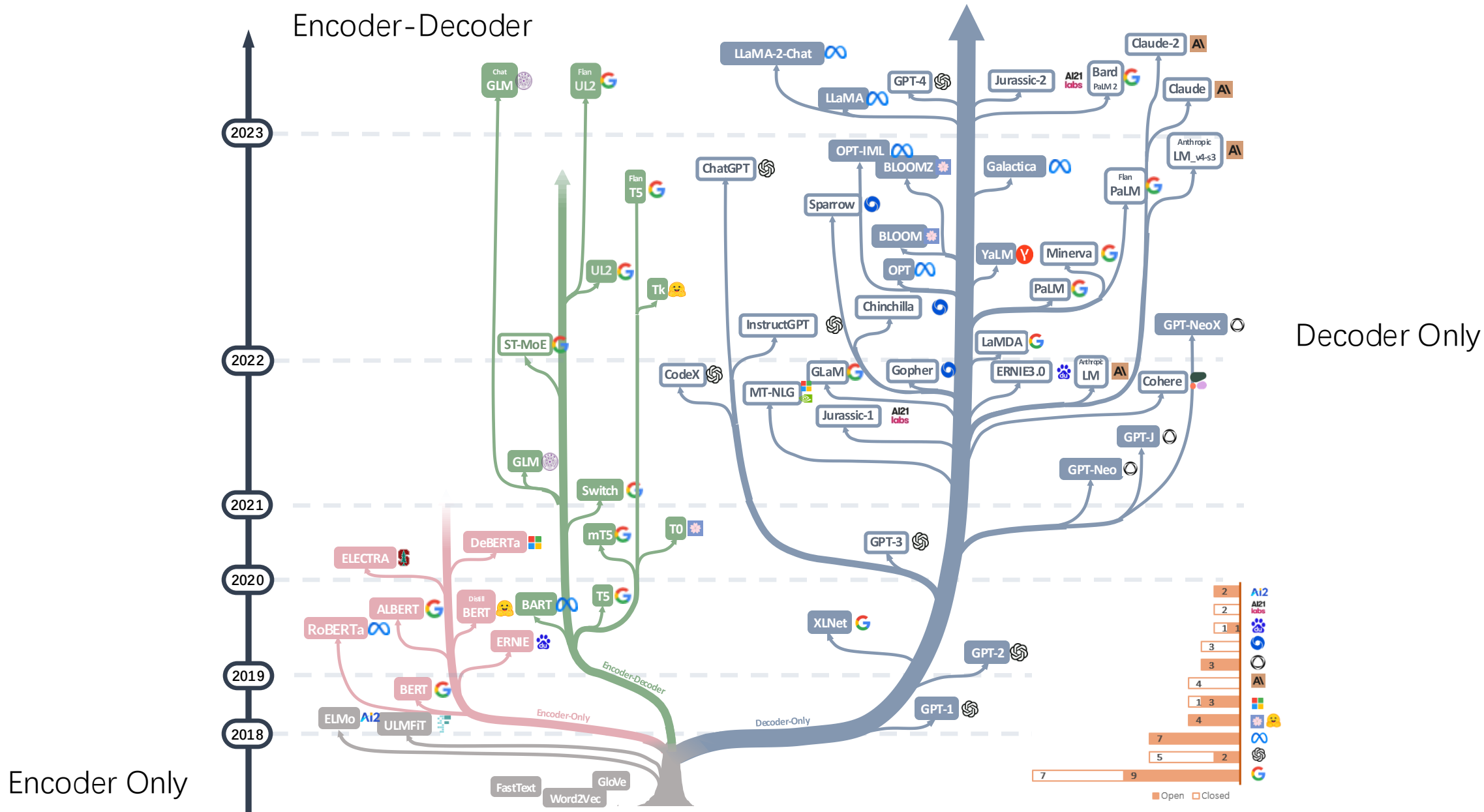
9688650

The original transformer paper used both an **encoder** and a **decoder**. Why?

Encoder used for **input** and the **decoder** was used for **output**.



Stopped Here





What is Chat GPT?

Chat: natural language system

✓ **G: Generatively** – Designed to model the creation of text

P: Pretrained – Trained on lots of naturally occurring data

✓ **T: Transformer** – A kind of neural network architecture

Chat GPT is just one example of a

✓ **Large Language Model (LLM)**



Generative Pre-training (Decoder only)

Single Passage of Text

We the People of the United States, in Order to form a more perfect Union, establish Justice, ...

Tune the model parameters to **maximize the likelihood** of the next token

Each token
is a training example

$\Pr(\text{"We"} \mid \text{""})$

$\Pr(\text{"the"} \mid \text{"We"})$

$\Pr(\text{"People"} \mid \text{"We the"})$

$\Pr(\text{"of"} \mid \text{"We the People"})$

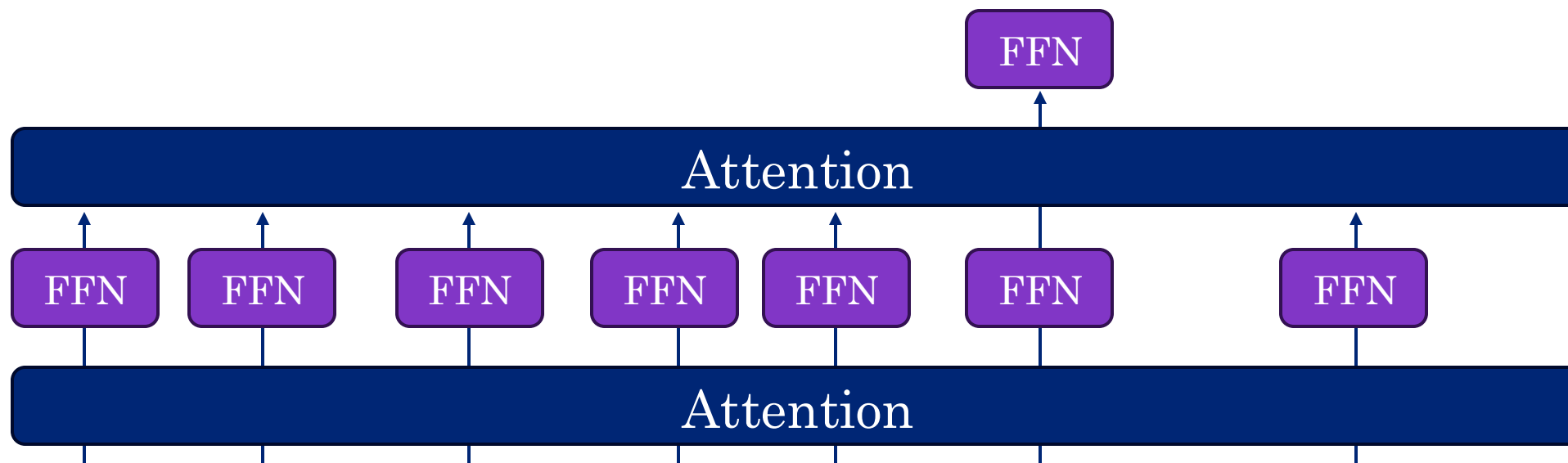
$\Pr(\text{"the"} \mid \text{"We the People of"})$

...

Masked Language Pre-Training (Encoders)



$\Pr(\text{"United"} \mid \text{"We the people of the MASK states"})$



We the people of the MASK states ...

Masking results in less learning signal and more difficult to train.



Pre-training on *Everything**

Train the model on a **large collection** of data to learn **generalizable patterns**

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

OpenAI
GPT3
Data Mix

Reddit
Links

Llama-3 “open-source” models trained on **15.6T tokens** from **an unknown data mix**.

- [405-billion parameter model](#) trained on 16K H100s (\$25K each)
- **39.3M GPU hours**

*Everything that is legal to use for training hopefully...



Demo

Decoder Transformer Notebook



9688650



9688650

Now you know

What is Chat GPT

There is still one more thing

Chat: natural language system

- ✓ **G: Generatively** – Designed to model the creation of text
- ✓ **P: Pretrained** – Trained on lots of naturally occurring data
- ✓ **T: Transformer** – A kind of neural network architecture

GPT alone can't chat!



Need to Teach Model to Follow Instructions (and be fun!)

- Generative pre-training **captures knowledge**
 - To finish the statement “Four score and seven years ago ...” you need to learn to memorize the text
- Resulting model predicts the rest of a statement.
 - “What is attorney client privilege?” the model might generate “Provide a concise answer using an example from class.”
- Use **Supervised Fine-Tuning** or **RLHF** to adjust “behavior” of model to **follow instructions** and **chat like a human**.

Fine-tuning



Running **additional training iterations** with a specific task

- The task differs from the original pre-training task
 - **New objective:** translate sentence, follow instruction
 - **New training data** from new source domain
- **Smaller learning rate** (as you get older you learn slower?)
 - Avoid “catastrophic forgetting” (new information causes forgetting pre-training information)



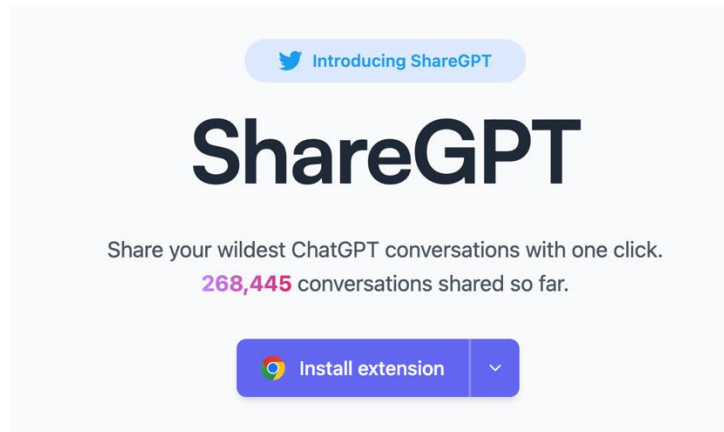
Vicuña

Open-source Instruction Fine-Tuned LLMs and LLM-as-a-judge

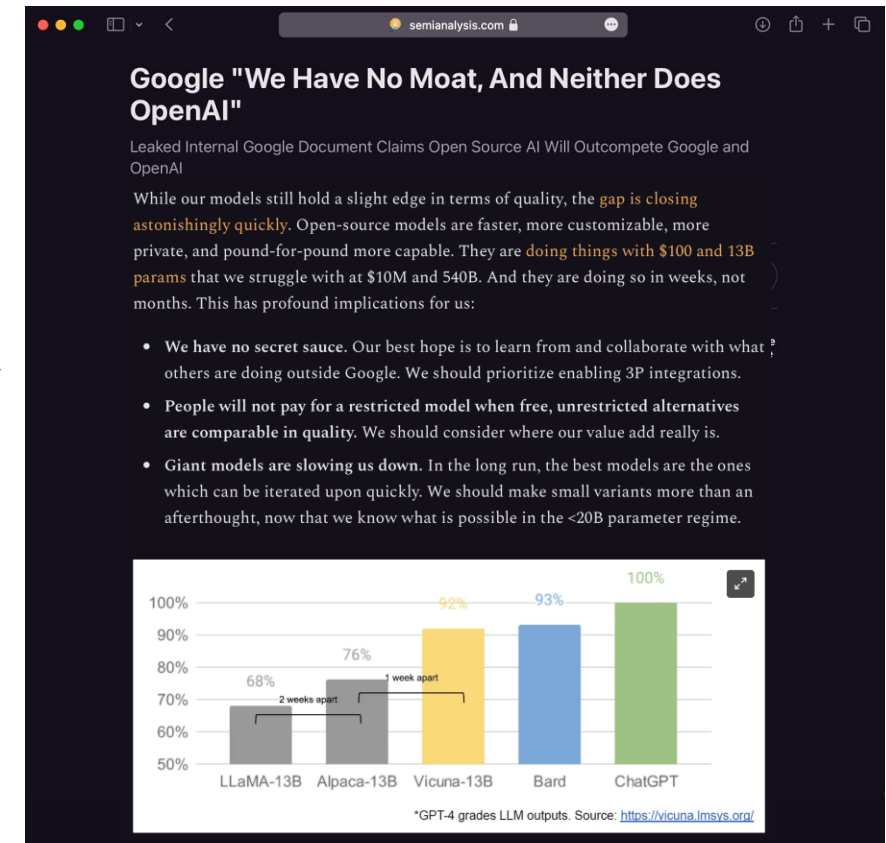


9688650

- First open-source model that was “comparable” to ChatGPT
 - Fine-tuned LLaMA-13B on ShareGPT Data



Tiny
High Quality Data
70K conversations
(~800MB)

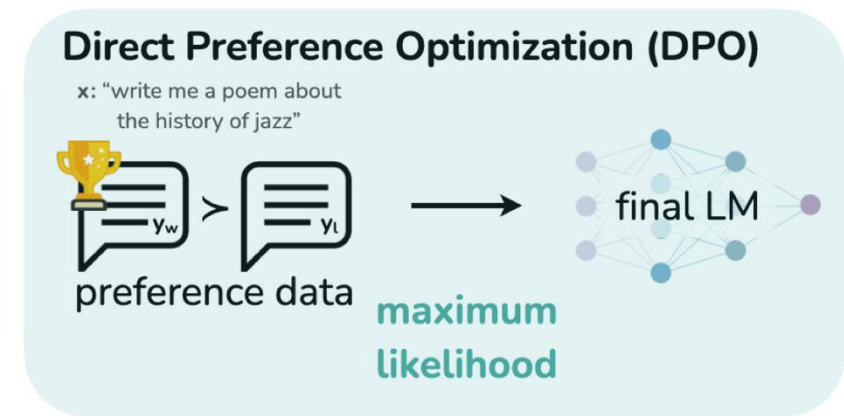
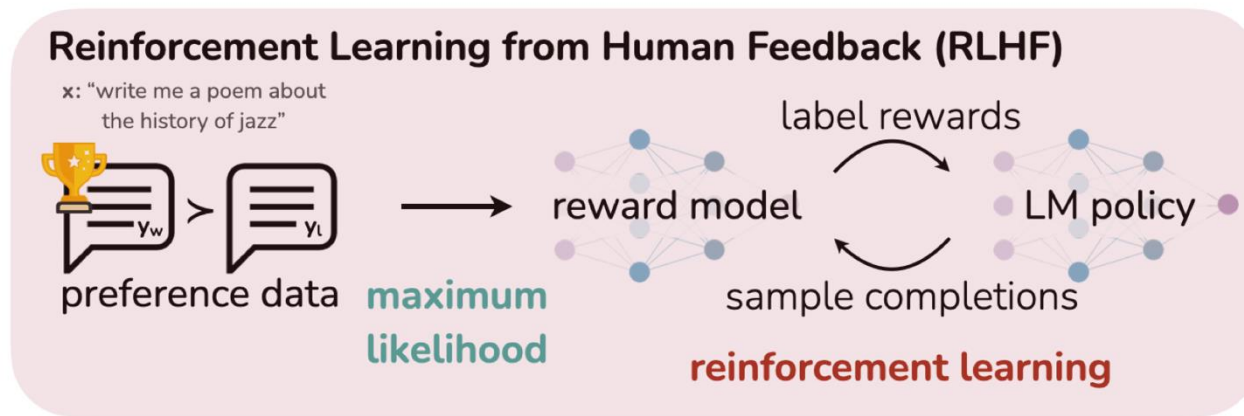


- Helped launch academic open-source **GenAI research**

Reward Optimization



Can further **align model with human preferences** using **human preference data**: *“this is better than that”*



Help to **make models more robust** and perform better in **safety situations**.



Now you know

What is Chat GPT

✓ **Chat**: natural language system

✓ **G: Generatively** – Designed to model the creation of text

✓ **P: Pretrained** – Trained on lots of naturally occurring data

✓ **T: Transformer** – A kind of neural network architecture

How are people using it?



Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

Zero-shot relies on model already
“knowing” how to complete the task.

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

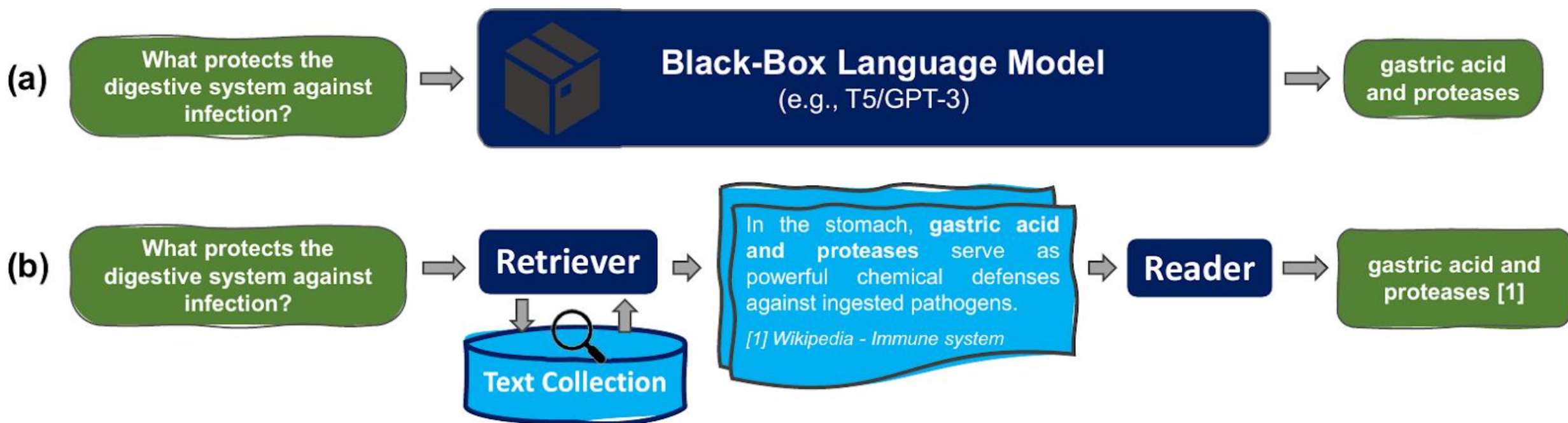
```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

In-context Learning

Retrieval Augmented Generation (RAG)



- Use external data to augment LLMs

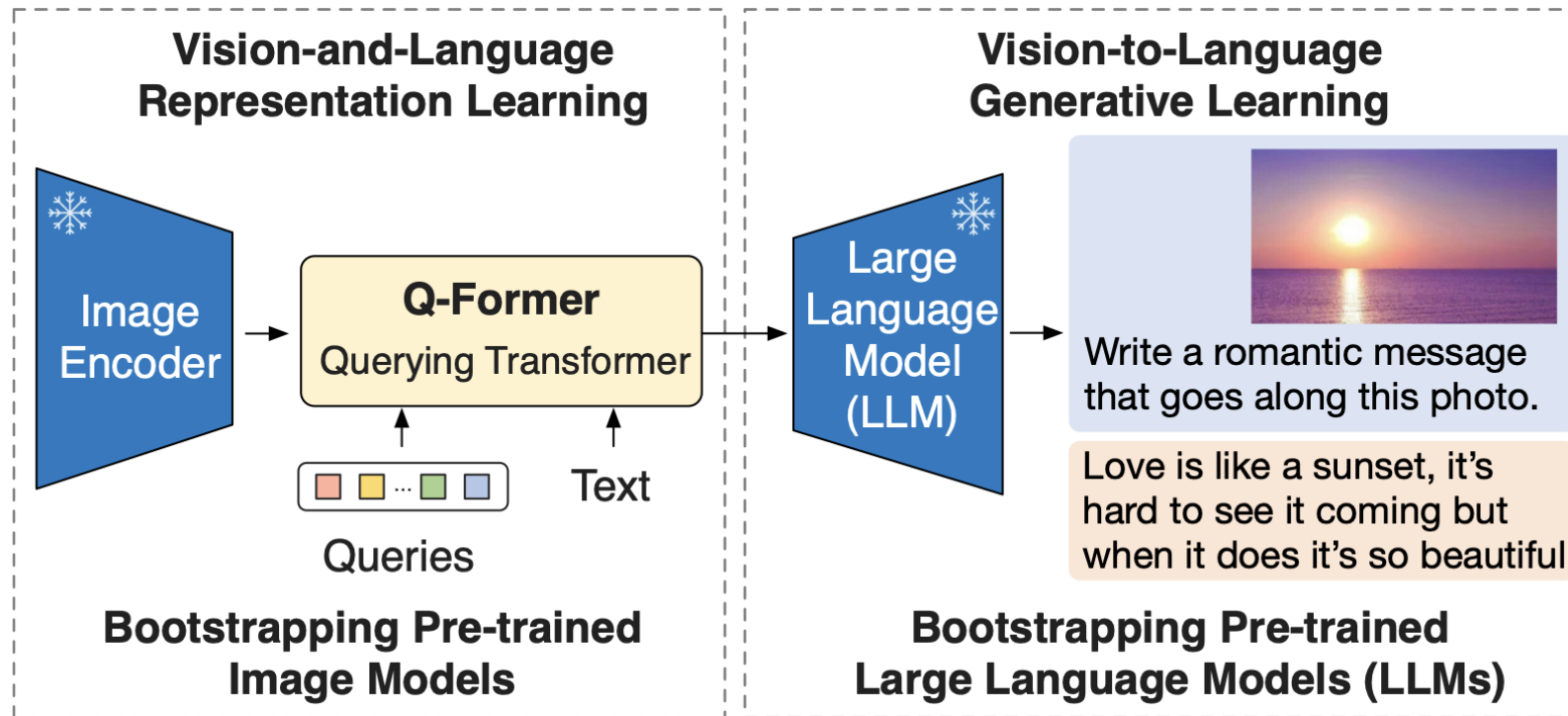


<https://acl2023-retrieval-lm.github.io>

Large Multi-Modal Models (LMMs)



- Combining with vision models to enable visual reasoning



Commercial and Open-Source Models



- **Use commercial services:** OpenAI, Google, Anthropic, ...
 - State-of-the-art accuracy and fast
 - Constantly changing black box
 - Often affordable: priced per token (1M tokens ~ \$10USD)
- **Use open-source models:** Llama3, Vicuña (mine!), Mixtral, ...
 - Often built from Meta's open-source Llama (1,2,3) models
 - Varying sizes (7B, 13B, 30B, 70B) and quantization levels (low bit precision)
 - **Variable accuracy and speed depends on hardware**
 - Bigger is more accurate and slower

Most organizations are using a mix of these technologies.

Lecture 21

Transformers and Language Modeling

Reading: *Chapter 10* in Bishop Deep Learning Textbook